

Termostat Digital

Student: Șișca Silviu-Claudiu

Anul: III, 2133;

Cuprins

1. Cerința proiectului.....	4
2. Alegerea senzorului de temperatură.....	6
2.1. Categorii majore de senzori de temperatură.....	6
2.2. Analiza senzorilor integrați analogici.....	6
2.3. Justificarea soluției.....	7
3. Arhitectura hardware a sistemului	8
3.1. Etajul de achiziție: Senzorul LM35	8
3.2. Etajul de condiționare: Amplificator neinvertor	9
3.3. Etajul de conversie și afișare: ADC0804.....	10
3.4. Simularea circuitului	12
4. Comanda digitală a modului LCD 16x2 prin stări logice	14
4.1. Principiu de funcționare	14
4.2. Instrucțiunile de control.....	14
4.3. Avantajele și dezavantajele modului LCD 16x2.....	16
4.3.1. Avantaje	16
4.3.2. Dezavantaje și limitări.....	16
4.4. Implementarea circuitului și rezultatele simulării	16
4.4.1. Secvența de inițializare a sistemului (RS = 0)	17
4.4.2. Procesul de afișare a datelor.....	17
5. Conversia tensiunii de la 9V la 5V	21
5.1. Stabilizatorul liniar	21
5.2. Convertor coborât în comutație (Buck)	23
5.3. Alegerea circuitului optim.....	25
6. Analiza microcontrolerelor	26
6.1. Intel 8051.....	26
6.2. ATmega328P	28
6.3. PIC16F877A.....	29
6.4. Concluzia analizei microcontrolerelor	30
7. Implementarea software a sistemului.....	31
7.1. Diagrama bloc de funcționare	31
7.2. Preluarea datelor de la ADC.....	33
7.3. Interpretarea datelor.....	33

7.4. Afișarea datelor pe LCD.....	36
7.5. Simulări	37
8. Implementarea sistemului de comandă	42
8.1. Circuitul de comandă.....	42
8.2. Logica de comandă a butoanelor	43
8.2.1. Conexiunea hardware.....	43
8.2.2. Implementarea Software	44
8.2.3. Gestiunea întreruperilor	45
8.3. Algoritmul de control si implementarea histerezei	46
8.4. Simulări	48
9. Cod C & Assembly	52
9.1. Cod C.....	52
9.2. Cod Assembly	55
10. Concluzii	64
11. Bibliografie	65

1. Cerința proiectului

Obiectivul acestui proiect este proiectarea și implementarea unui termostat de cameră bazat pe arhitectura microcontrolerului 8051. Proiectul abordează atât partea hardware cât și cea software, cu o cerință specială de implementarea a codului în C și Assembly.

Pe parcursul acestui proiect se vor realiza următoarele cerințe:

1. Realizarea unei analize comparative pentru alegerea unui senzor de temperatură. Procesul impune identificarea a 5 categorii majore de senzori termici, selectarea și argumentarea celei mai potrivite categorii pentru arhitectura sistemului, urmată de compararea a 5 senzori diferiți din respectiva categorie. Senzorul ales are de îndeplinit următoarele specificații: intervalul minim de temperatură este 0°C - 45°C , iar precizia să fie în intervalul $\pm 0.2^{\circ}\text{C}$ - $\pm 0.5^{\circ}\text{C}$
2. Proiectarea unui circuit electric pentru măsurarea temperaturii utilizând senzorul LM35 și convertorul analogic-digital ADC0804. Sistemul necesită un etaj intermediar de condiționare a semnalului, capabil să adapteze ieșirea senzorului astfel încât intervalul termic de măsurare 0°C - 40°C să fie mapat pe întreaga gamă dinamică a convertorului. Calibrarea finală a circuitului trebuie să asigure o corespondență liniară a ieșirii digitale între limitele 00 (pentru 0°C) și FF (pentru 40°C). Valoarea obținută la ieșirea convertorului va fi afișată în format hexazecimal pe 2 display-uri cu 7 segmente.
3. Realizarea unui studiu asupra funcționalității unui ecran cu cristale lichide (LCD) alfanumeric de tip 16x2 (LM016L). Obiectivul principal este înțelegerea protocolului de comunicație paralelă la nivel hardware și validarea acestuia prin comandarea manuală a afișajului (utilizând exclusiv componente de tip Logic State) în mediul de simulare Proteus, pentru a afișa pe primul rând "Sisca Silviu" iar pe al doilea rând "Grupa 2133".
4. Realizarea unui studiu comparativ structurat pe două direcții principale, punând accent pe alegerea soluțiilor optime. Prima parte a studiului presupune analiza posibilităților de a converti o tensiune continuă de 9V într-o tensiune stabilizată de 5V, prezentând schemele și argumentând cea mai bună abordare practică. A doua parte constă în studierea și compararea unor microcontrolere (incluzând arhitectura clasică 8051), pentru a stabili varianta cea mai potrivită și eficientă.
5. Proiectarea și implementarea software a unui sistem cu microcontroler capabil să citească o temperatură, să o proceseze și să o afișeze cu o precizie de o zecimală. Microcontrolerul trebuie să îndeplinească următoarele funcționalități:
 - Preluarea datelor: trebuie să comunice cu convertorul analog-digital (ADC) pentru a prelua datele brute
 - Interpretarea datelor: valoarea digitală brută trebuie prelucrată matematic de către microcontroler pentru a fi convertită în grade Celsius, scalate corect pentru intervalul dorit de temperatură
 - Afișarea datelor: rezultatul obținut în urma interpretării trebuie formatat și trimis către modulul de afișare (LCD)

-
6. Implementarea unui sistem de comandă cu releu, care este controlat de un microcontroler 8051. Capabilitățile pe care trebuie să le îndeplinească sistemul sunt:
- Modificarea temperaturii de referință prin intermediul unor butoane.
 - Comanda releului prin intermediul microcontrolerului.
 - Implementarea unui algoritm de control cu histereză pentru comutarea releului.

2. Alegerea senzorului de temperatură

Temperatura este una dintre cele mai frecvent monitorizate mărimi fizice în sistemele de control automat. Pentru a converti această mărime într-un semnal electric ce poate fi prelucrat ulterior de un microcontroler, este necesară utilizarea unui traductor specific. Alegerea senzorului de temperatură dictează în mod direct complexitatea etajului hardware de condiționare a semnalului și arhitectura software a proiectului.

2.1. Categoriile majore de senzori de temperatură

Pentru a stabili soluția optimă, a fost realizată o analiză a pieței componentelor electronice. Senzorii de temperatură se împart în 5 mari categorii, în funcție de principiul fizic pe care se bazează:

1. **Termistoare (NTC/PTC):** sunt componente pasive a căror rezistență variază odată cu temperatura, deși sunt ieftine și au un timp de răspuns bun, variația lor este puternic neliniară.
2. **Detectoare de temperatură cu rezistență (RTD):** acestea se bazează pe variația rezistenței metalelor pure, precum platina. Oferă o precizie și stabilitate industrială excepțională, dar semnalul generat este foarte slab, impunând utilizarea unor circuite de amplificare și condiționare scumpe și complexe.
3. **Termocupluri:** sunt formate din joncțiunea a două metale diferite care generează o tensiune proporțională cu diferența de temperatură. Sunt ideale pentru măsurarea temperaturilor extreme.
4. **Senzori integrați digitali:** sunt circuite integrate care includ intern elementul sensibil și convertorul analog-digital. Oferă valoarea temperaturii direct sub formă de pachet de date digitale, comunicând cu microcontrolerul prin protocoale specifice.
5. **Senzori integrați analogici:** sunt circuite integrate care furnizează la ieșire o tensiune analogică perfect proporțională cu temperatura, fiind deja liniarizate și calibrate.

Pentru a evita complexitatea liniarizării termistoarelor sau a amplificării RTD-urilor a fost selectată categoria senzorilor integrați analogici, deoarece aceștia generează un semnal de tensiune stabil, compatibil direct cu intrarea convertorului ADC0804 ales pentru acest proiect.

2.2. Analiza senzorilor integrați analogici

În cadrul categoriei selectate, au fost analizate 5 variante de senzori des utilizați în proiectarea sistemelor:

1. **LM35:** este un circuit integrat de precizie, calibrat în grade Celsius, cu un factor de scară de $10\text{mV}/^{\circ}\text{C}$. Acesta poate opera liniar pe un domeniu extins și oferă o precizie tipică de $\pm 0.5^{\circ}\text{C}$ la 25°C
2. **LM335:** ieșirea este calibrată în grade Kelvin ($10\text{mV}/\text{K}$).
3. **LM34:** este un senzor de înaltă precizie, dar calibrat pentru Fahrenheit ($10\text{mV}/^{\circ}\text{F}$).

4. **TMP36:** furnizează o tensiune liniară proporțională cu temperatura în grade Celsius, totuși precizia sa tipică este de $\pm 1^{\circ}\text{C}$ la temperatura camerei și poate ajunge la $\pm 2^{\circ}\text{C}$ pe întregul domeniu.
5. **MCP9700:** este un senzor de putere mică, ideal pentru aplicații pe baterie. Prezintă o eroare tipică de $\pm 2^{\circ}\text{C}$.

2.3. Justificarea soluției

În urma analizei comparative, s-a ajuns la concluzia că senzorul LM35 este soluția optimă care îndeplinește toate cerințele proiectului. Acesta garantează o acuratețe de $\pm 0.5^{\circ}\text{C}$ la temperatura camerei și operează fără probleme în plaja $0^{\circ}\text{C} - 100^{\circ}\text{C}$, depășind intervalul minim necesar de $0^{\circ}\text{C} - 45^{\circ}\text{C}$. De asemenea, calibrarea este în grade Celsius și generează 0V la 0°C .

3. Arhitectura hardware a sistemului

Pentru preluarea temperaturii ambientale și transmiterea acesteia către microcontrolerul 8051, a fost proiectat un lant de achiziție din trei etaje principale: senzorul de temperatură, etajul de condiționare a semnalului și etajul de conversie analog-digitală. Figura 3.1 prezintă schema electrică a acestui circuit.

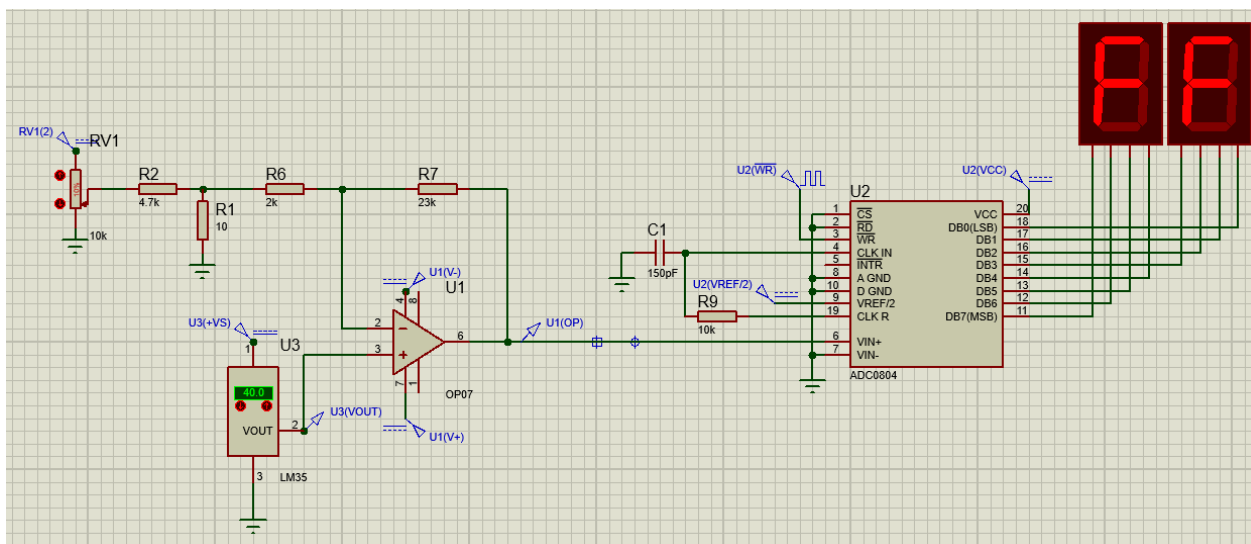


Figura 3.1: Sistem de achizitie, conditionare și afisare a temperaturii

3.1. Etajul de achizitie: Senzorul LM35

Pentru măsurarea temperaturii se utilizează senzorul integrat LM35. Acesta are tensiunea de ieșire direct proporțională cu temperatura în grade Celsius.

Conform foii de catalog:

- Factorul de scalare liniar este 10mV/°C
- Gama de temperatură: -55°C - 150°C
- Tensiunea de alimentare: 4V – 30V

Senzorul LM35 are 3 terminale:

- Pin 1 ($+V_S$) – reprezintă alimentarea senzorului
- Pin 2 (V_{out}) – reprezintă ieșirea analogică a senzorului
- Pin 3 (GND) – reprezintă masa circuitului.

În circuitul proiectat, senzorul este alimentat cu +5V.

Având în vedere cerințele de proiectare, gama de temperaturi ce trebuie măsurată este cuprinsă între 0°C și 40°C. Folosind factorul de scalare, se determină limitele tensiunii analogice furnizate la pinul de ieșire:

Pentru $T_{min} = 0^{\circ}\text{C}$

$$V_{out} = 0^{\circ}\text{C} * 10 \frac{\text{mV}}{^{\circ}\text{C}} = 0\text{V}$$

Pentru $T_{max} = 40^{\circ}\text{C}$

$$V_{out} = 40^{\circ}\text{C} * 10 \frac{\text{mV}}{^{\circ}\text{C}} = 0.4\text{V}$$

La ieșirea etajului de achiziție, tensiunea va varia liniar în intervalul $0\text{V} - 0.4\text{V}$.

3.2. Etajul de condiționare: Amplificator neinversor

Pentru condiționarea semnalului, se va implementa un amplificator neinversor cu compensare de offset. La intrarea etajului este prezentă o tensiune cuprinsă între 0V și 0.4V , iar ieșirea trebuie cuprinsă în intervalul $0\text{V} - 5\text{V}$.

Astfel, pentru a mapa intervalul $0\text{V} - 0.4\text{V}$ la intervalul $0\text{V} - 5\text{V}$, este necesară o amplificare liniară. Factorul de amplificare (A_v) este calculat astfel:

$$A_v = \frac{\Delta V_{out}}{\Delta V_{in}} = \frac{5 - 0}{0.4 - 0} = 12.5$$

Pentru implementarea circuitului, s-a optat pentru un amplificator operațional de precizie (OP07) într-o configurație neinversoare. Semnalul de la senzor (V_{in}) este aplicat pe borna neinversoare (+), iar pe borna inversoare (-) se află o rețea de reacție formată din rezistoarele R_7 și R_6 , combinată cu un circuit de compensare a offset-ului.

Funcția de transfer teoretică a acestui circuit este dată de relația:

$$V_{out} = V_{in} \left(1 + \frac{R_7}{R_6} \right) - V_x \frac{R_7}{R_6}$$

Unde V_x este tensiunea de offset generată de divizorul rezistiv, iar $(1 + \frac{R_7}{R_6})$ este factorul de amplificare A_v .

$$A_v = \left(1 + \frac{R_7}{R_6} \right) = 12.5$$

$$R_7 = 11.5 R_6$$

Se aleg $R_6 = 2\text{k}\Omega$, $R_7 = 23\text{k}\Omega$.

Deși la intrarea etajului valoarea minimă teoretică este 0V și amplificatorul este unul de precizie (cu offset foarte mic), în funcționarea circuitului real pot apărea erori, provenite fie din toleranțele componentelor, fie din imperfecțiunea amplificatorului. Astfel s-a decis implementarea circuitului de compensare de offset, care este compus din potențiometrul RV1 ($10\text{k}\Omega$) conectat la o sursă de tensiune $V_{RV1} = 5\text{V}$, și rezistențele $R_2 = 4.7\text{k}\Omega$ și $R_1 = 10\Omega$.

Deoarece potențiometrul este conectat între $V_{RV1} = 5\text{V}$ și GND, tensiunea la cursor depinde de poziția acestuia:

$$V_{cursor} = k * V_{RV1} = k * 5V$$

$$k \in [0, 1]$$

În nodul format de R_2 și R_1 apare tensiunea V_x , care are formula:

$$V_x = \frac{R_1}{R_1 + R_2} * V_{cursor} = 2.12 * 10^{-3} * V_{cursor}$$

$$V_{x_max} = 2.12 * 10^{-3} * 5 = 10.61mV$$

Astfel, din relația:

$$V_{out} = V_{in} \left(1 + \frac{R_7}{R_6} \right) - V_x \frac{R_7}{R_6}$$

deducem că valoarea maximă pe care o poate compensa circuitul este:

$$\Delta V_{out,max} = V_{x_max} \frac{R_7}{R_6} = 10.61mV * \frac{23k\Omega}{2k\Omega} = 122mV$$

Un aspect important în implementarea acestui etaj este alimentarea amplificatorului operațional. Pentru a permite ieșirii să atingă valorile dorite, amplificatorul este alimentat cu o tensiune simetrică de $\pm 12V$. Utilizarea unei alimentări unipolare nu ar fi fost adecvată, deoarece OP07 nu este un amplificator “rail-to-rail”.

3.3. Etajul de conversie și afișare: ADC0804

Pentru ultima etapă, este necesară conversia semnalului analogic care vine de la etajul anterior într-unul digital, ce urmează a fi afișat pe cele 2 display-uri cu 7 segmente. Pentru această sarcină s-a ales utilizarea unui convertor analog-digital ADC0804, un convertor cu aproximații succesive pe 8 biți.

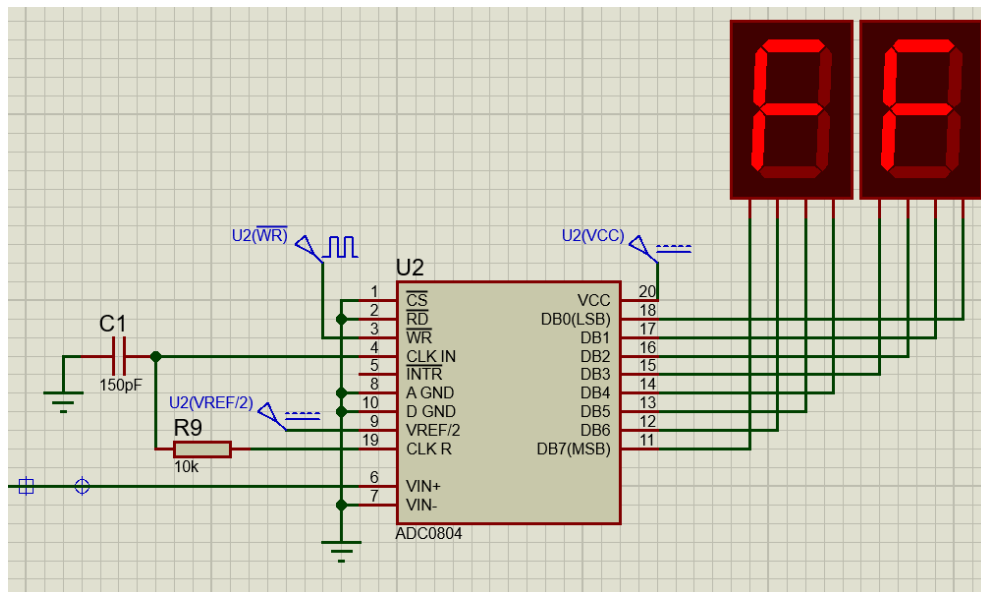


Figura 3.2: Implementarea etajului de conversie și afișare

Semnalul analogic provenit de la etajul de amplificare (cuprins între 0V – 5V) este aplicat pe intrarea analogică pozitivă, pinul 6 (V_{in+}). Pentru a realiza o măsurătoare referențiată la masă, intrarea negativă, pinul 7 (V_{in-}), este conectată la GND. La pinul 20 (V_{cc}) este conectată o sursă de tensiune de 5V, astfel valoarea maximă a scalei va fi 5V, iar la pinul 9 ($V_{REF}/2$) este conectată o sursă de tensiune de 2.5V.

Având în vedere că circuitul operează pe 8 biți și are $V_{REF} = 5V$, rezoluția se calculează astfel:

$$V_{LSB} = \frac{V_{full-scale}}{2^n} = \frac{5V}{256} \approx 19.5mV$$

Pentru executarea conversiilor analog-digitale, ADC0804 necesită un semnal de ceas. În această implementare s-a optat pentru utilizarea oscilatorului intern al integratului, a cărui structură este prezentată în Figura 3.3.

9.2.1.2.5 Clocking Option

The clock for the ADC can be derived from the CPU clock or an external RC can be added to provide self-clocking. The CLK IN (pin 4) makes use of a Schmitt trigger as shown in Figure 35.

$$f_{CLK} = \frac{1}{RC \ln \left[\left(\frac{V_{CC} - V_{T-}}{V_{CC} - V_{T+}} \right) \left(\frac{V_{T+}}{V_{T-}} \right) \right]}$$

$$R \approx 10 \text{ k}\Omega$$

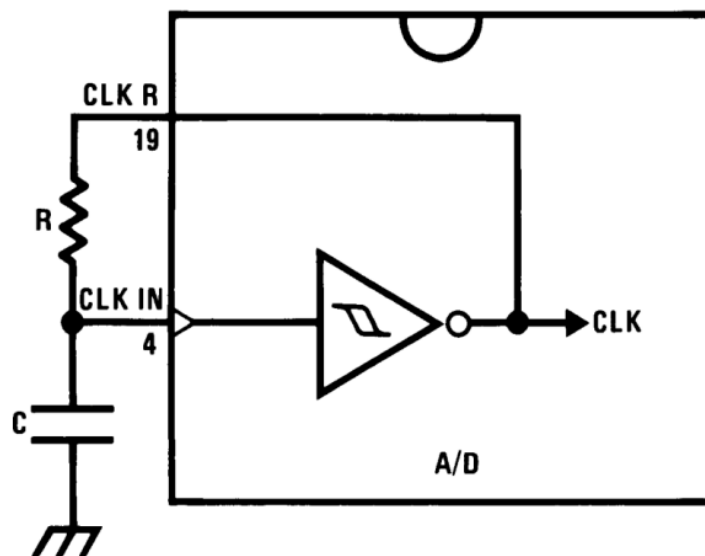


Figure 35. Self-Clocking the ADC

Figura 3.3: Configurația de auto-generare a semnalului de ceas pentru ADC0804 (Sursa: Datasheet Texas Instruments ADC080x)

Din datasheet extragem valorile tipice pentru V_{T+} , 3.1V, și V_{T-} , 1.8V. În circuit s-a ales $C_1 = 150pF$ și $R_9 = 10k\Omega$. Astfel se calculează frecvența semnalului de ceas:

$$f_{CLK} = \frac{1}{10^4 * 150 * 10^{-12} * \ln \left[\left(\frac{5 - 1.8}{5 - 3.1} \right) \left(\frac{3.1}{1.8} \right) \right]}$$

$$f_{CLK} \simeq 626kHz$$

Pentru a finaliza o conversie completă, acest circuit integrat necesită între 66 și 73 perioade de ceas. Astfel, se poate calcula perioada de timp cea mai defavorabilă pentru o conversie:

$$T_{conv} = \frac{N_{cicluri}}{f_{CLK}} = \frac{73}{626 \text{ kHz}} \simeq 116 \mu\text{s}$$

Pinii 1 (Chip Select) și 2 (Read) sunt conectați permanent la nivelul logic LOW (GND). Această configurație menține convertorul activ în permanență.

Pinul 3 (Write) este comandat de un semnal de ceas extern de tip dreptunghiular cu frecvența de 50Hz (perioada de 20ms). Fiecare tranziție din starea HIGH în starea LOW pe acest pin declanșează o nouă conversie.

Rezultatul conversiei este disponibil la pinii de date digitali (DB0 – DB7). Pentru a afișa rezultatul în format hexazecimal pe cele 2 display-uri cu 7 segmente, am împărțit cei 8 biți în 2 grupuri de câte 4 biți fiecare : grupul inferior DB0 – DB3 și grupul superior DB4 - DB7.

3.4. Simularea circuitului

Pentru a valida funcționarea corectă a circuitului, au fost testate 3 puncte din domeniul de temperatură:

- **limita superioară (40°C)** – tensiunea atinge valoarea maximă de 5 V la intrarea convertorului, generându-se valoarea FF (Figura 3.4).
- **mijlocul intervalului (20°C)** - tensiunea atinge valoarea de 2.5 V la intrarea convertorului, generându-se valoarea 7F (Figura 3.5).
- **limita inferioară (0°C)** - tensiunea atinge valoarea minimă de aproximativ 0 V la intrarea convertorului, generându-se valoarea 00 (Figura 3.6).

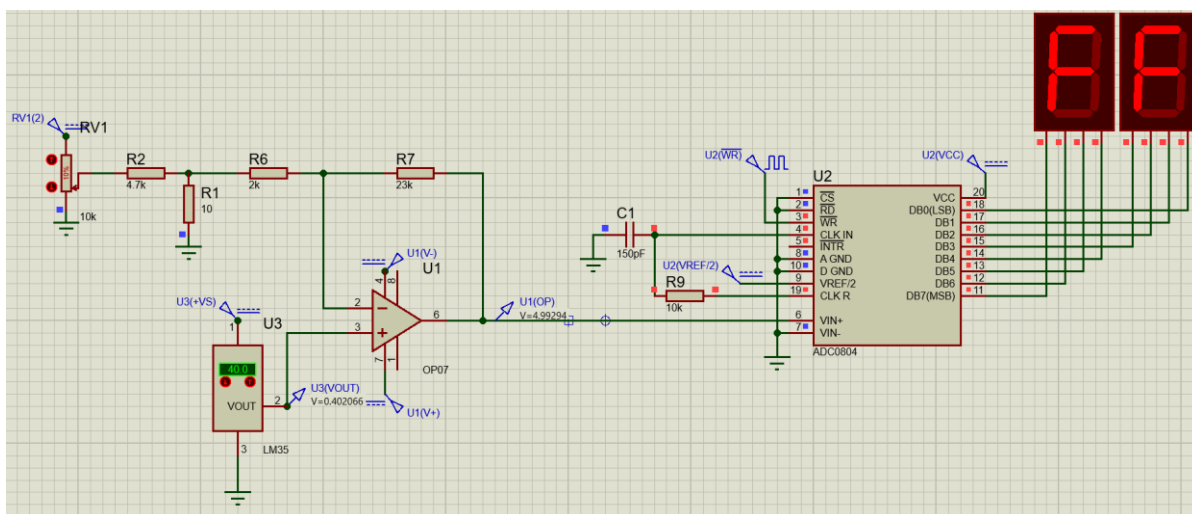


Figura 3.4: Simularea circuitului pentru limita superioară de temperatură

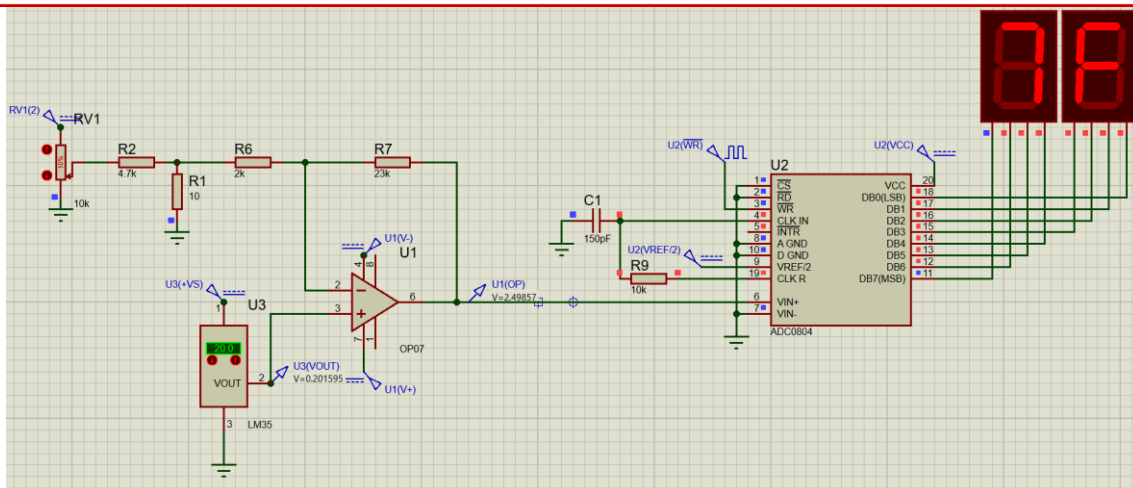


Figura 3.5: Simularea circuitului pentru valoarea de mijloc a intervalului de temperatură

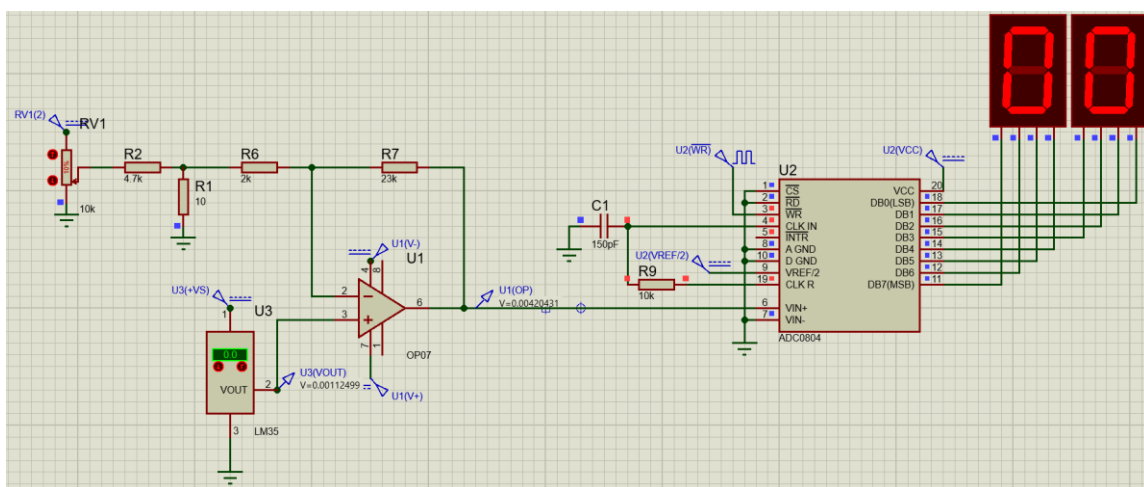


Figura 3.6: Simularea circuitului pentru limita inferioară de temperatură

4. Comanda digitală a modulului LCD 16x2 prin stări logice

4.1. Principiu de funcționare

Pentru implementare s-a utilizat modulul LCD LM016L, bazat pe controlerul intern HD44780. Acest ecran are capacitatea de a afișa simultan 32 de caractere ASCII, organizate pe 2 rânduri a câte 16 coloane. Comunicarea cu acest modul se realizează printr-o magistrală paralelă de date și un set de pini de control.

Nr. Pin	Simbol	Stare	Funcționalitate	
1	V_{SS}	-	0V	Alimentare
2	V_{DD}	-	5V	
3	V_{EE} / V_O	-	-	
4	RS	H/L	Selectează registrul intern de destinație al datelor de pe magistrală. H: Modul de date L: Modul de comandă	
5	R/W	H/L	H: Citește Date (Modul LCD → Magistrală) L: Scrie Date (Modul LCD ← Magistrală)	
6	E	H, H→L	Semnal Enable, pentru ca LCD-ul să citească starea pinilor de date, se aplică un impuls logic pozitiv. Datele sunt preluate pe frontul descrescător al semnalului.	
7	DB0	H/L	Magistrala bidirecțională pe 8 biți	
8	DB1	H/L		
9	DB2	H/L		
10	DB3	H/L		
11	DB4	H/L		
12	DB5	H/L		
13	DB6	H/L		

4.2. Instrucțiunile de control

Afișajul LCD are un set de instrucțiuni predefinite. Atunci când pinul RS este în starea L (0), controlerul intern (HD44780) este în modul de comandă, iar orice valoare pusă pe magistrala de date (DB0-DB7) nu este afișată ca text, ci este executată ca o comandă internă care schimbă starea sau configurația ecranului.

Cele mai importante instrucțiuni utilizate în procesul de inițializare și configurare sunt:

1. Function Set (0x38): Aceasta este prima comandă trimisă și stabilește parametrii fundamentali de interfațare.

Valoarea 0x38 setează: interfață pe 8 biți, afișare pe 2 linii și font de 5x7 puncte.

2. Display ON/OFF Control (0x0E): Controlează vizibilitatea datelor și a cursorului.

Valoarea 0x0E: pornește ecranul, face cursorul vizibil și dezactivează clipirea cursorului.

3. Entry Mode Set (0x06): Defiște comportamentul cursorului după scrierea unui caracter.

Valoarea 0x06 stabilește: incrementarea automată a adresei, ceea ce face ca cursorul să se miște la dreapta după fiecare literă scrisă, și dezactivează deplasarea (shift) întregului ecran.

4. Clear Display (0x01): Curăță ecranul de simboluri și readuce cursorul la poziția inițială

5. Set DDRAM Address: Permite selectarea manuală a poziției cursorului

Valoarea 0x80: mută cursorul la începutul rândului 1

Valoarea 0xC0: mută cursorul la începutul rândului 2

Un tablou complet al tuturor instrucțiunilor disponibile, împreună cu configurația biților și timpii de execuție necesari este prezentat în Figura 4.1.

Command	Code										Description	Execution Time		
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0				
Clear Display	0	0	0	0	0	0	0	0	0	1	Clears the display and returns the cursor to the home position (address 0).	82μs-1.64ms		
Return Home	0	0	0	0	0	0	0	0	0	1	*	Returns the cursor to the home position (address 0). Also returns a shifted display to the home position. DD RAM contents remain unchanged.	40μs-1.64ms	
Entry Mode Set	0	0	0	0	0	0	0	0	1	I/D	S	Sets the cursor move direction and enables/disables the display.	40μs	
Display ON/OFF Control	0	0	0	0	0	0	0	1	D	C	B	Turns the display ON/OFF (D), or the cursor ON/OFF (C), and blink of the character at the cursor position (B).	40μs	
Cursor & Display Shift	0	0	0	0	0	0	1	S/C	R/L	*	*	Moves the cursor and shifts the display without changing the DD RAM contents.	40μs	
Function Set	0	0	0	0	0	1	DL	N\$	F	*	#	Sets the data width (DL), the number of lines in the display (L), and the character font (F).	40μs	
Set CG RAM Address	0	0	0	0	1	A _{CG}					Sets the CG RAM address. CG RAM data can be read or altered after making this setting.		40μs	
Set DD RAM Address	0	0	0	1	A _{DD}					Sets the DD RAM address. Data may be written or read after making this setting.		40μs		
Read Busy Flag & Address	0	1	BF	AC								Reads the BUSY flag (BF) indicating that an internal operation is being performed and reads the address counter contents.	1μs	
Write Data to CG or DD RAM	1	0	Write Data										Writes data into DD RAM or CG RAM.	46μs
Read Data from CG or DD RAM	1	1	Read Data										Reads data from DD RAM or CG RAM.	46μs
	I/D = 1: Increment I/D = 0: Decrement S = 1: Accompanies display shift. S/C = 1: Display shift S/C = 0: cursor move R/L = 1: Shift to the right. R/L = 0: Shift to the left. DL = 1: 8 bits DL = 0: 4 bits N = 1: 2 lines N = 0: 1 line F = 1: 5x10 dots F = 0: 5 x 7 dots BF = 1: Busy BF = 0: Can accept data # Set to 1 on 24x4 modules \$ With KS0072 is Address Mode.										DD RAM: Display data RAM CG RAM: Character generator RAM A _{CG} : CG RAM Address A _{DD} : DD RAM Address Corresponds to cursor address. AC: Address counter Used for both DD and CG RAM address.		Execution times are typical. If transfers are timed by software and the busy flag is not used, add 10% to the above times.	

Figura 4.1: Set de instrucțiuni și comenzi pentru LCD cu HD44780

4.3. Avantajele și dezavantajele modulului LCD 16x2

Utilizarea unui ecran cu cristale lichide de tip 16x2 aduce o serie de beneficii tehnice semnificative, dar implică și anumite compromisuri.

4.3.1. Avantaje

- Afișarea alfanumerică este unul dintre avantajele majore, deoarece LCD-ul permite afișarea întregului set de caractere ASCII. Acest lucru permite afișarea unor mesaje clare.
- Consumul energetic redus (aproximativ 1mA, maxim 3mA) și cost de achiziție foarte scăzut.
- Controlerul HD44780 este un standard universal, acest lucru înseamnă că protocolul de comunicare este același indiferent de producător, făcând ecranul ușor de înlocuit sau integrat.

4.3.2. Dezavantaje și limitări

- Consum ridicat de resurse I/O (pini). În configurația cu modul de transfer pe 8 biți, modulul necesită 11 pini conectați la microcontroler (3 pini de comandă, 8 pini de date).
- Limitări grafice, ecranul este conceput pentru a afișa simultan doar 32 de caractere ASCII, fiecare caracter fiind format dintr-o matrice de 5x7 pixeli. Din acest motiv, ecranul nu poate afișa fonturi personalizate sau animații complexe.
- Inițializare complexă și timpi de așteptare. Controlerul HD44780 este un dispozitiv relativ lent, necesită o rutină strictă de inițializare la pornire și respectarea timpilor de execuție a instrucțiunilor (40μs – 1.64ms).

4.4. Implementarea circuitului și rezultatele simulării

În această etapă, s-a realizat transpunerea teoretică a protocolului de comunicație HD44780 într-o simulare în mediul Proteus. Controlul afișajului s-a efectuat manual, utilizând componente de tip Logic State pentru a simula semnalele digitale trimise de un microcontroler. În Figura 4.2 este prezentat circuitul folosit.

Notă: Deși în implementarea fizică pinul 3 (VEE) necesită un divizor rezistiv (potențiomtru) pentru reglarea contrastului cu cristale lichide, în mediul de simulare Proteus acest pin a fost lăsat neconectat, modelul software al LCD-ului gestionând automat contrastul optim pentru vizualizare.

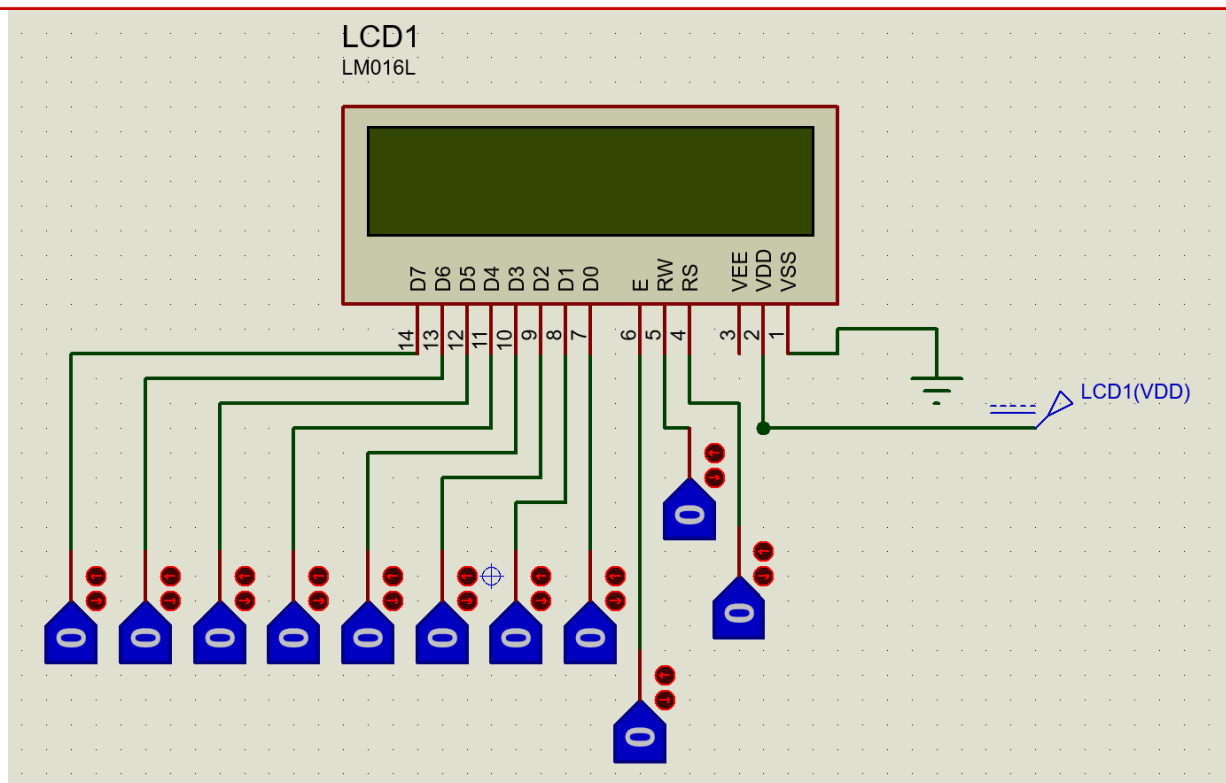


Figura 4.2: Implementarea circuitului LCD 16x2

4.4.1. Secvența de inițializare a sistemului (RS = 0)

Înainte de scrierea oricărui caracter, afișajul necesită o procedură de configurare. Fiecare instrucțiune a fost trimisă setând magistrala pe valorile din tabelul de mai jos și aplicând un impuls (0→1→0) pe pinul Enable (E).

Este important de menționat că, între transmiterea a două instrucțiuni consecutive, este necesară respectarea unor timpi de așteptare specificați în datasheet-ul controlerului HD44780.

Instrucțiune	Hex	Binar
Function Set	0x38	0011 1000
Display ON/OFF	0x0E	0000 1110
Clear Display	0x01	0000 0001
Entry Mode	0x06	0000 0110

4.4.2. Procesul de afișare a datelor

După finalizarea inițializării, pinul RS a fost setat pe nivelul logic 1. S-au introdus succesiv codurile ASCII pentru textul dorit, conform tabelelor de mai jos. Procesul de simulare a fost documentat în 3 etape:

1. Scrierea pe primul rând: S-au trimis caracterele “Sisca Silviu”, rezultatul parțial fiind prezentat în Figura 4.3.

ASCII	Hex	Binar
S	0x53	0101 0011
i	0x69	0110 1001
s	0x73	0111 0011
c	0x63	0110 0011
a	0x61	0110 0001
(spațiu)	0x20	0010 0000
S	0x53	0101 0011
i	0x69	0110 1001
l	0x6C	0110 1100
v	0x76	0111 0110
i	0x69	0110 1001
u	0x75	0111 0101

2. Schimbarea rândului. Pentru a trece de pe primul rând pe cel de-al doilea, s-a revenit scurt la RS=0 pentru a trimite comanda de adresare 0xC0 (Set DDRAM Address). Această etapă de re poziționare a cursorului este ilustrată în Figura 4.4.

Instrucțiune	Hex	Binar
Set DDRAM Address	0xC0	1100 0000

3. Finalizarea afișării. Se setează RS=1, și se introduc caracterele pentru “Grupa 2133”, rezultatul final fiind surprins în Figura 4.5.

ASCII	Hex	Binar
G	0x47	0100 0111
r	0x72	0111 0010
u	0x75	0111 0101
p	0x70	0111 0000
a	0x61	0110 0001
(spațiu)	0x20	0010 0000
2	0x32	0011 0010
1	0x31	0011 0001
3	0x33	0011 0011
3	0x33	0011 0011

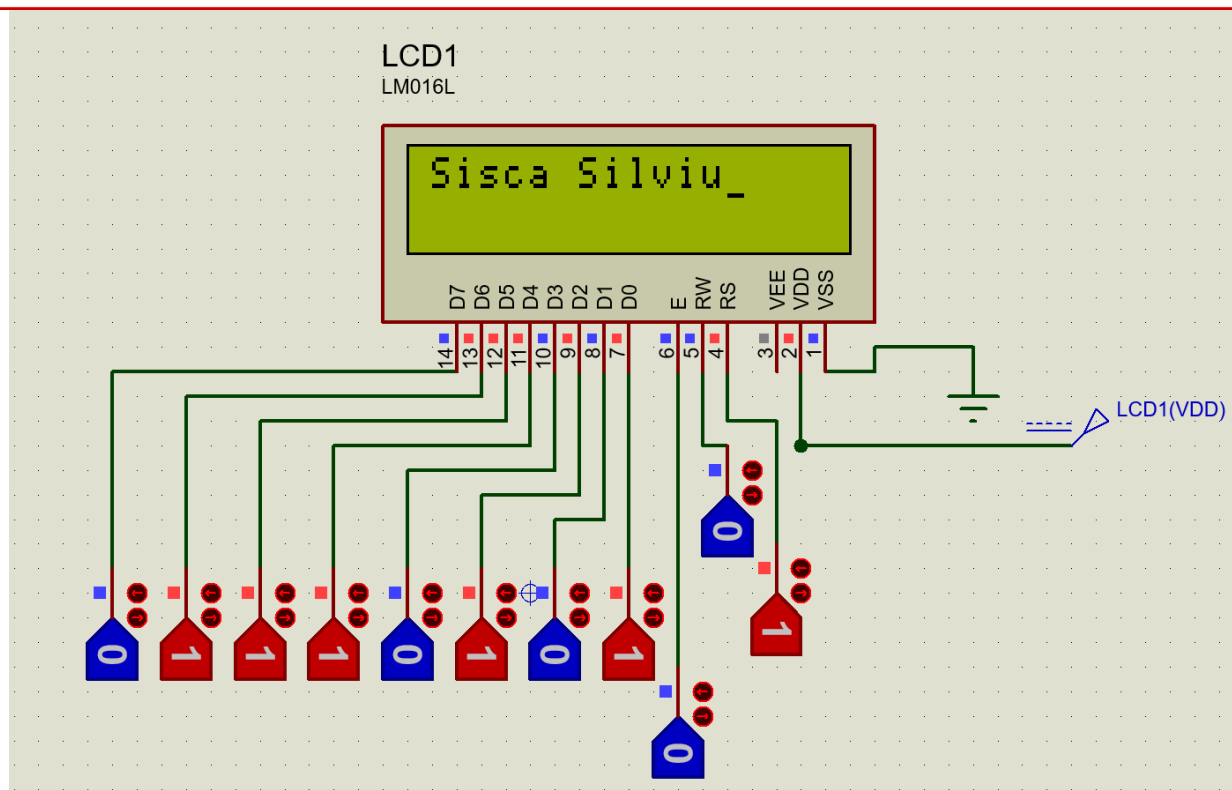


Figura 4.3: Scrierea pe primul rând – Rezultat parțial

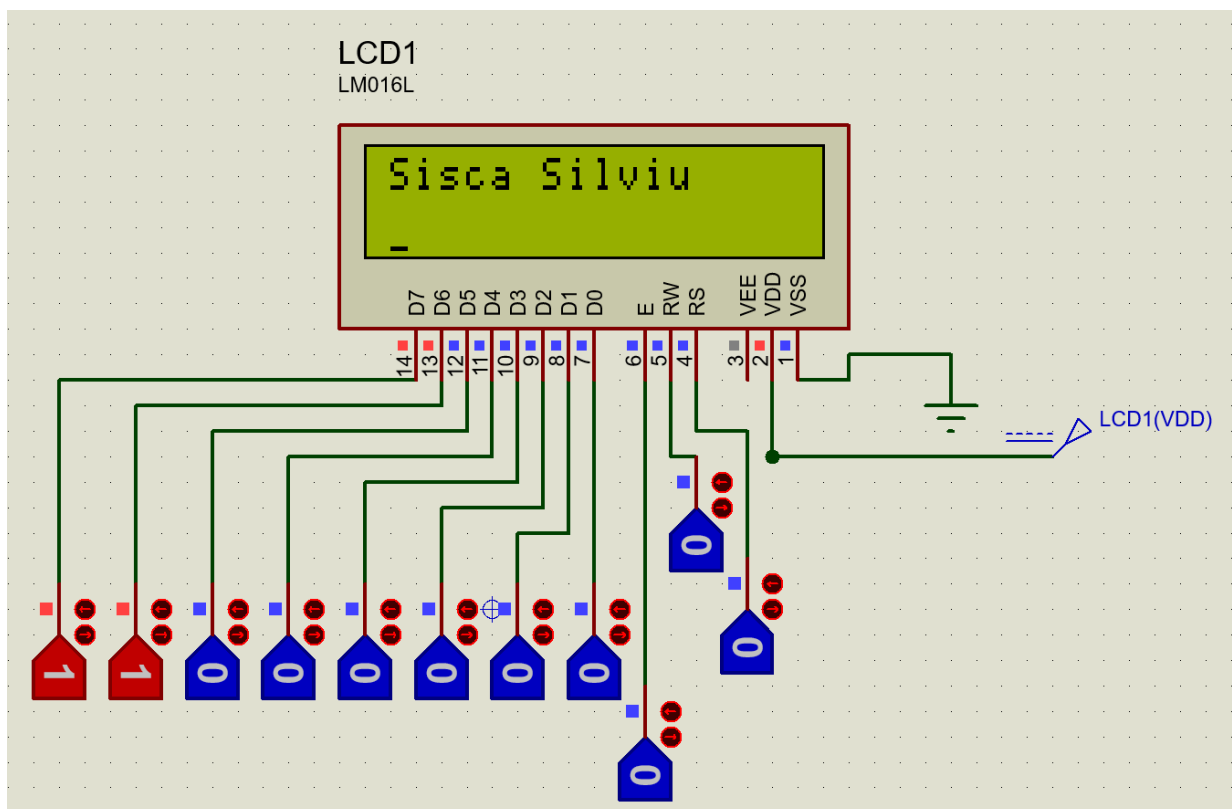


Figura 4.4: Schimbarea rândului – Rezultat parțial

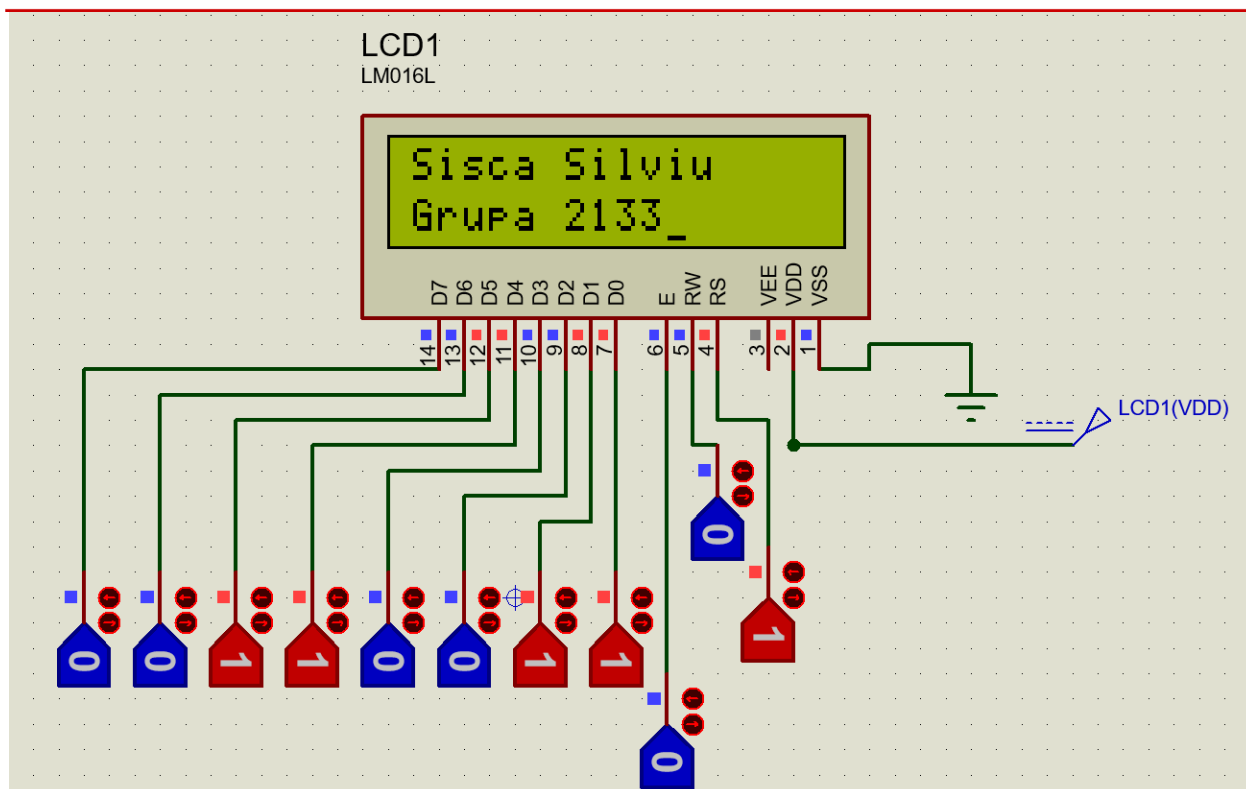


Figura 4.5: Finalizarea afișării – Rezultat final

5. Conversia tensiunii de la 9V la 5V

Pentru alimentarea părții logice a unui termostat digital (microcontroler, senzori, afișaj, etc.) dintr-o sursă de 9V, este necesară obținerea unei tensiuni continue și stabilizate de 5V. Pentru realizarea acestei conversii putem alege dintre cele două abordări principale: stabilizatoare liniare și convertoare coborâtoare în comutație (Buck).

5.1. Stabilizatorul liniar

Pentru realizarea stabilizatorului liniar s-a utilizat circuitul prezentat în Figura 5.1, arhitectura acestuia bazându-se pe o buclă de reacție negativă, menită să mențină o tensiune constantă la ieșire, indiferent de variațiile tensiunii de intrare sau ale sarcinii.

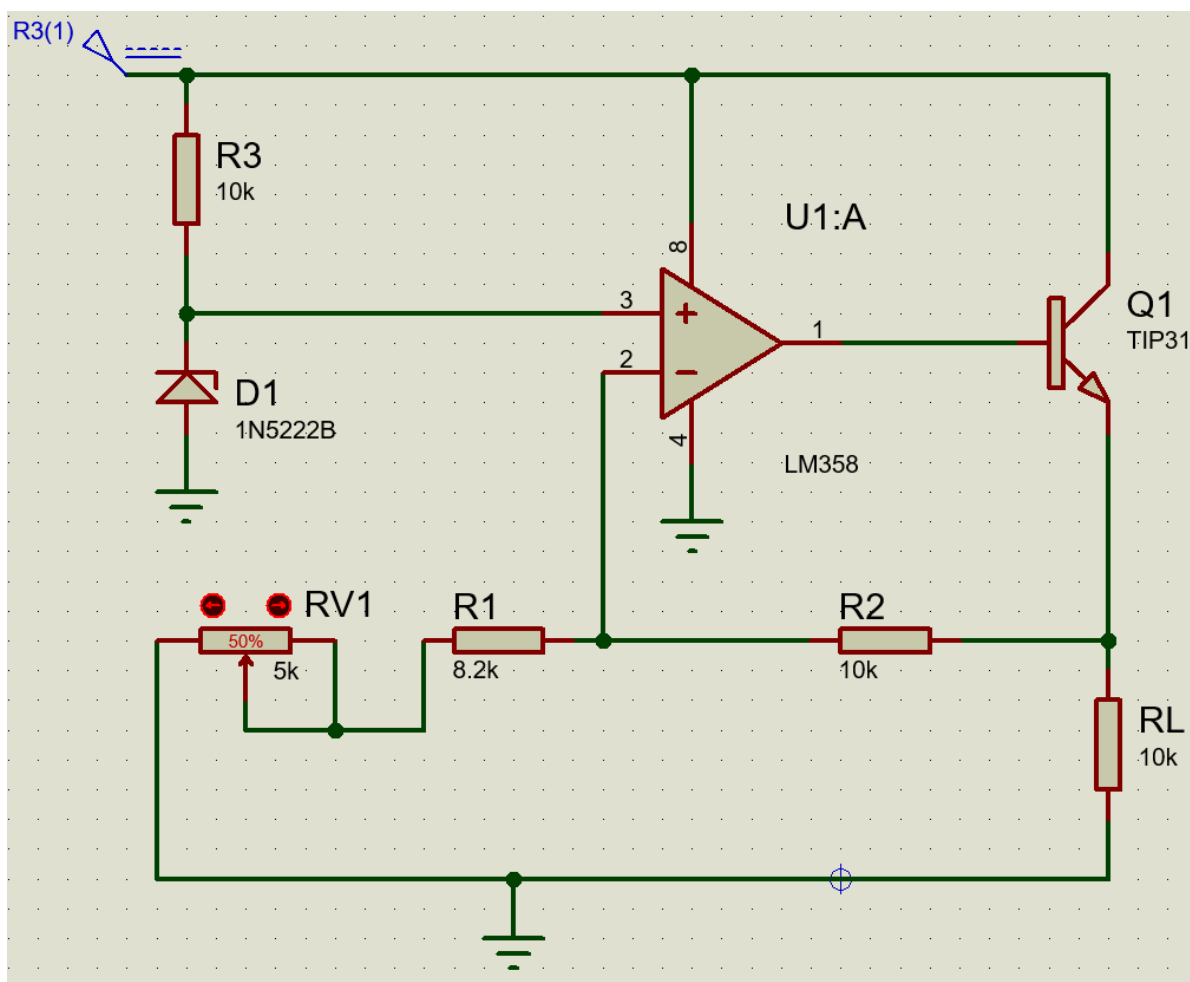


Figura 5.1: Schema stabilizatorului liniar

Dioda Zener 1N5222B are rolul de referință de tensiune, aceasta fiind invers polarizată prin rezistența R_3 , și generează o tensiune stabilă de $V_{REF} = 2.5V$.

Tranzistorul TIP31 funcționează ca element regulator serie, el fiind conectat între sursa de intrare și sarcină. Acesta se comportă ca o rezistență dinamică controlată în tensiune, funcționând exclusiv în regiunea activă normală.

Amplificatorul LM358 compară fracțiunea din tensiunea de ieșire cu referința fixă. Orice diferență detectată este amplificată și aplicată bazei tranzistorului TIP31 pentru a corecta ieșirea.

Rezistențele R_1 , R_2 și potențiometrul RV_1 alcătuiesc rețeaua de reacție. Utilizarea potențiometrului RV_1 înseriat cu R_1 permite un reglaj fin, necesar pentru a compensa toleranțele de fabricație ale diodei Zener și ale rezistențelor.

Tensiunea de ieșire a stabilizatorului este determinată de configurația de amplificator neinvertor a buclei de reacție. Considerând amplificatorul operațional ideal:

$$V_- = V_+$$

tensiunea pe pinul neinvertor fiind:

$$V_+ = V_Z = V_{REF}$$

iar relația tensiunii de pe pinul invertor:

$$V_- = V_{out} \frac{R_1 + RV_{1set}}{R_1 + R_2 + RV_{1set}}$$

rezultă că tensiunea de ieșire este:

$$V_{out} = V_{REF} \left(1 + \frac{R_2}{R_1 + RV_{1set}} \right)$$

Pentru un $V_{out} = 5V$, cu $V_{REF} = 2.5V$, rezultă ca raportul rezistențelor este unitar:

$$\frac{R_2}{R_1 + RV_{1set}} = 1$$

Valorile alese pentru rezistențe sunt:

$$R_1 = 8.2 \text{ k}\Omega$$

$$R_2 = 10 \text{ k}\Omega$$

$$RV_1 = 5 \text{ k}\Omega$$

În simulare, s-a realizat ajustarea RV_1 astfel încât suma $(R_1 + RV_{1set})$ să fie egală cu R_2 , iar rezultatele să fie cele dorite. Figura 5.2 ilustrează simularea circuitului proiectat.

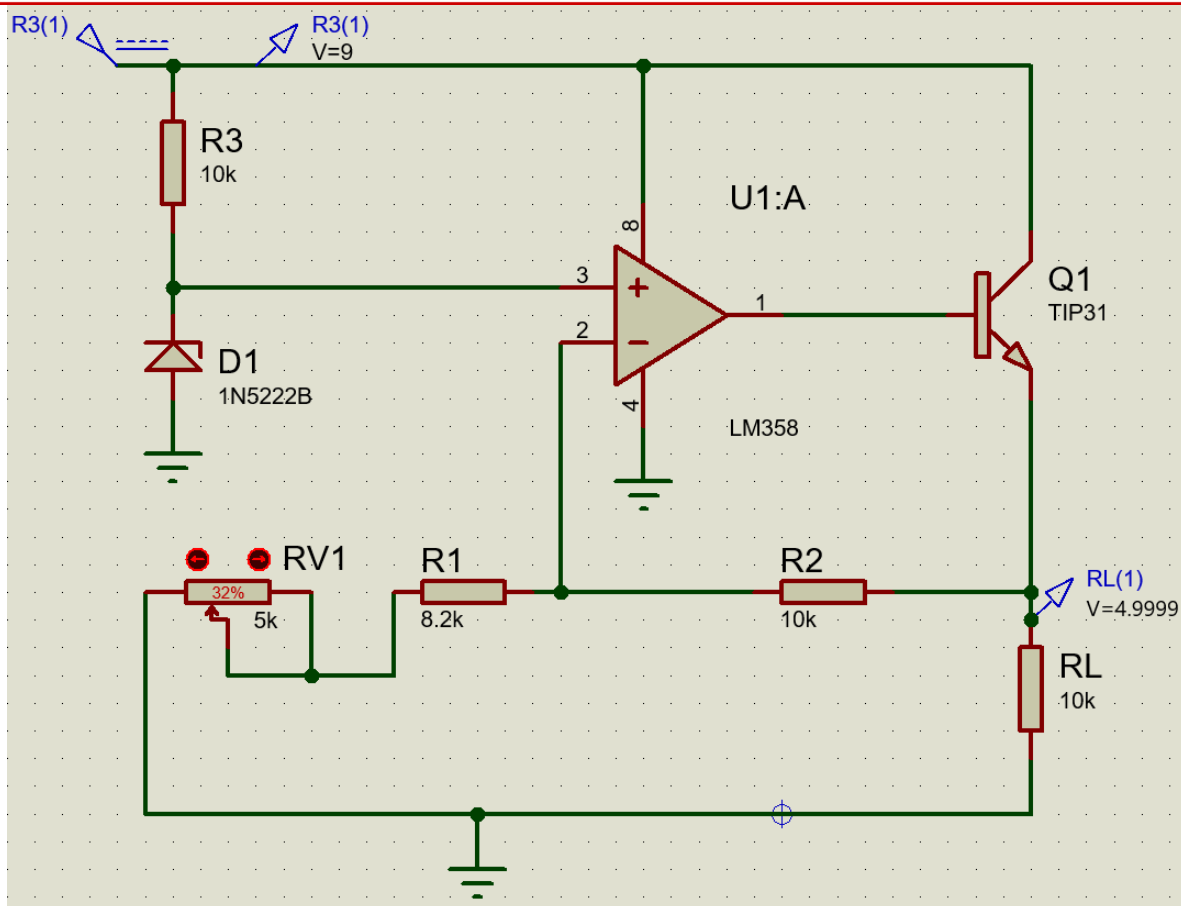


Figura 5.2: Simularea stabilizatorului liniar

Pentru a valida fiabilitatea soluției, s-a calculat eficiența și puterea disipată luând în calcul un consum ipotetic de maxim 70mA (microcontroler + LCD):

$$\eta = \frac{V_{out}}{V_{in}} = \frac{5V}{9V} \approx 55.5\%$$

$$P_d = (V_{in} - V_{out}) * I = (9V - 5V) * 0.07A = 0.28W$$

5.2. Convertor coborât în comutație (Buck)

Spre deosebire de topologia convertorului liniar care disipă energia suplimentară, un convertor de tip Buck funcționează în regim de comutație. Acesta transferă energia de la sursă la sarcină în impulsuri, utilizând un tranzistor care se deschide și se închide la o frecvență înaltă, și componente (o bobină și un condensator) care filtrează și stochează această energie. Figura 5.3 ilustrează schema de principiu a unui convertor Buck.

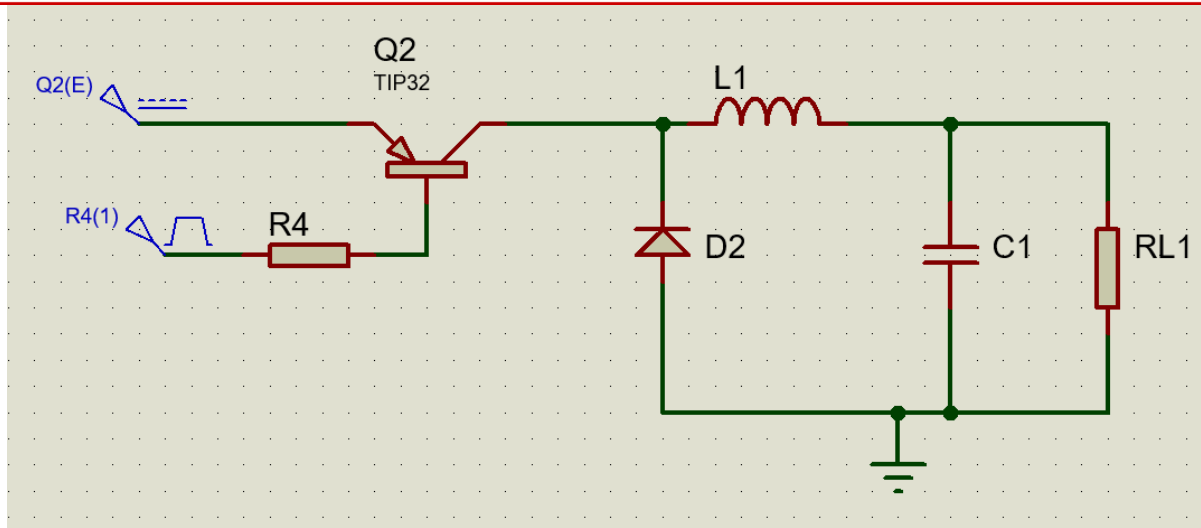


Figura 5.3: Schema convertor de tip Buck

Tranzistorul PNP, TIP32, este conectat cu emitorul la sursa de intrare, și are rolul de a “tăia” tensiunea continuă, fiind comandat în bază de un semnal de tip PWM.

Când tranzistorul este ON, curentul prin bobină crește liniar, înmagazinând energie sub formă de câmp magnetic.

Când tranzistorul devine OFF, câmpul magnetic al bobinei se prăbușește, inversându-și polaritatea. Dioda se deschide instantaneu, oferind o cale de închidere a circuitului pentru curentul din bobină spre sarcină.

Condensatorul absoarbe pulsațiile de curent, transformând impulsurile într-o tensiune de 5V.

Spre deosebire de stabilizatorul liniar care disipă surplusul de energie, convertorul Buck reglează tensiunea modificând raportul de timp dintre starea ON și starea OFF a tranzistorului. Acest raport se numește factor de umplere (D – Duty Cycle), iar pentru calcularea valorii este nevoie de V_{out} ($= 5V$), V_{in} ($= 9V$) și η care este estimat 90%:

$$D = \frac{V_{out}}{V_{in} * \eta} = \frac{5V}{9V * 0.9} \approx 61.7\%$$

Deoarece convertorul lucrează în regim de comutație, tensiunea și curentul de ieșire nu sunt perfect continue, ci prezintă o variație ciclică numită pulsație (ripple). Pentru a garanta buna funcționare a microcontrolerului, aceste pulsații trebuie menținute sub anumite limite. S-a ales un ripple de curent de 30% din curentul de sarcină, pe care îl considerăm ipotetic 70mA.

$$\Delta i_L = 0.30 * I_{out} = 0.30 * 0.07A = 0.021A$$

Frecvența de comutație aleasă este $f_s = 20kHz$, valoare adecvată pentru tranzistorul TIP32. Inductanța minimă se calculează din relația:

$$\Delta i_L = \frac{(V_{in} - V_{out}) * D}{f_s * L}$$

$$L = \frac{(V_{in} - V_{out}) * D}{f_s * \Delta i_L} = \frac{(9 - 5) * 0.617}{20000 * 0.021} \approx 5.87mH$$

Se va alege valoarea standard de 6.8mH.

Condensatorul C absoarbe ripple-ul de curent, menținând ripple-ul de tensiune la ieșire sub 1% din V_{out} ($\Delta V_{out} = 50mV$):

$$C = \frac{\Delta i_L}{8 * f_s * \Delta V_{out}} = \frac{0.021}{8 * 20000 * 0.050} \approx 2.63\mu F$$

Se va alege valoarea standard de 10 μF .

Spre deosebire de stabilizatorul liniar, convertorul Buck nu disipă surplusul de tensiune sub formă de căldură. Eficiența acestuia fiind de aproximativ 90%.

$$P_d = V_{out} * I_{out} * (1 - \eta) = 5V * 0.07A * (1 - 0.9) \approx 0.035W$$

5.3. Alegerea circuitului optim

Decizia finală privind etajul de alimentare trebuie să se bazeze pe un compromis între eficiența energetică, complexitatea de implementare și calitatea tensiunii furnizate. Analizând cele 2 topologii propuse, se pot trage următoarele concluzii:

- **Eficiența și regimul termic:** Convertorul Buck este superior la acest aspect, având o eficiență de aproximativ 90% și o putere disipată de doar 0.035W. Totuși, la un consum redus estimat de 70mA, stabilizatorul liniar disipă doar 0.28W. Această putere termică este suficient de mică pentru a nu necesita un radiator.
- **Calitatea tensiunii furnizate:** Convertoarele în comutație generează un ripple de tensiune la ieșire, care se suprapune pe tensiunea de alimentare de 5V. În contrast, stabilizatorul liniar furnizează o tensiune continuă stabilă, fără perturbații.

Deși convertorul Buck este mai eficient din punct de vedere energetic, circuitul care va fi alimentat (ADC, LM35 etc.) necesită o tensiune stabilă, fără ripple, rezultând astfel că stabilizatorul liniar este alegerea optimă.

6. Analiza microcontrolerelor

Studiul se concentrează pe o analiză comparativă între arhitectura clasică Intel 8051, și soluțiile moderne de tip RISC (Reduced Instruction Set Computer). Obiectivul principal este identificarea microcontrolerului care oferă cel mai bun echilibru între performanță, implementare și cost.

Microcontrolerelor care vor fi analizate se găsesc în tabelul de mai jos:

#	Microcontroler	Familia/Arhitectură	Producător
1	8051	MCS-51 (CISC)	Intel
2	ATmega328P	AVR (RISC)	Microchip Technology
3	PIC16F877A	PIC16 (RISC)	Microchip Technology

6.1. Intel 8051

Microcontrolerul 8051 reprezintă una dintre cele mai cunoscute arhitecturi de tip CISC pe 8 biți. Configurația fizică a pinilor, care evidențiază porturile de intrare/ieșire disponibile, este ilustrată în Figura 6.1 .

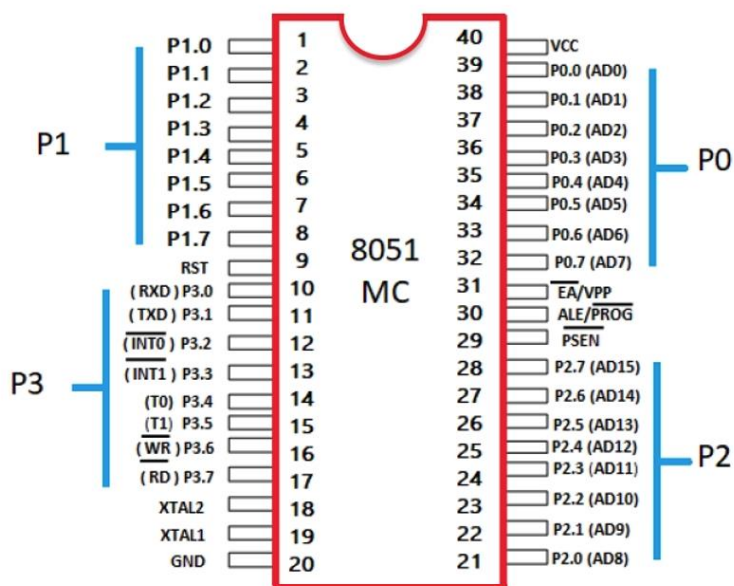


Figura 6.1: Pini microcontroler 8051

Funcțiile principale ale pinilor microcontrolerului 8051:

- VCC: pin pentru tensiunea de alimentare
- GND: pin pentru masă
- Port0 (P0.0 – P0.7): port de intrare/ieșire care funcționează și ca magistrală multiplexată pentru date și adrese de ordin inferior
- Port1 (P1.0 – P1.7): port de intrare/ieșire
- Port2 (P2.0 – P2.7): port de intrare/ieșire care funcționează și ca magistrală multiplexată pentru date și adrese de ordin superior

- Port3 (P3.0 – P3.7): port de intrare/ieșire, în care fiecare pin deține o funcție specială: comunicație serială (RXD, TXD), întreruperi externe (INT0, INT1), intrări pentru timere (T0, T1) și semnale pentru scrierea/citirea memoriei externe (WR, RD)
- RST: semnalul de resetare
- ALE/PROG (Address Latch Enable/ Program Pulse): funcția ALE este utilizată pentru a reține adresele de ordin inferior pe durata accesării memoriei externe. Funcția PROG este folosită la aplicarea impulsurilor necesare pentru programarea memoriei EPROM interne
- PSEN (Program Strobe Enable) : reprezintă semnalul de validare a citirii din memoria de program externă. Când acesta este în starea LOW, un octet reprezentând codul instrucțiunii este plasat pe magistrala de date
- XTAL1, XTAL2: conexiunile pentru generatorul de ceas
- EA/VPP (External Access Enable/Programming Supply Voltage)

Câteva specificații relevante ale 8051:

- Arhitectură pe 8 biți: ALU, registrele interne și magistrala de date sunt proiectate să proceseze date în format de 8 biți
- Memorie program: 4KB de memorie internă pentru stocarea codului executabil, poate fi extinsă extern până la 64 KB
- Memorie de date (RAM): 128 B, poate fi extinsă extern până la 64 KB
- Timere/Contoare: două timere pe 16 biți (Timer 0 și Timer 1) care pot fi folosite pentru generarea de temporizări precise sau numărarea evenimentelor externe
- Comunicație Serială: un port serial (UART) full-duplex
- Frecvența de ceas: de obicei 12 MHz
- Porturi I/O: 4 porturi paralele cu câte 8 pini fiecare

Un dezavantaj critic al 8051 în proiectul de față este lipsa unui Convertor Analog-Digital (ADC) integrat. Deoarece senzorul de temperatură LM35 furnizează un semnal analogic, utilizarea lui 8051 impune adăugarea unui ADC extern.

6.2. ATmega328P

Microcontrolerul Atmega328P face parte din familia AVR și se bazează pe o arhitectură de tip RISC pe 8 biți.

Configurația fizică a pinilor este ilustrată în Figura 6.2 .

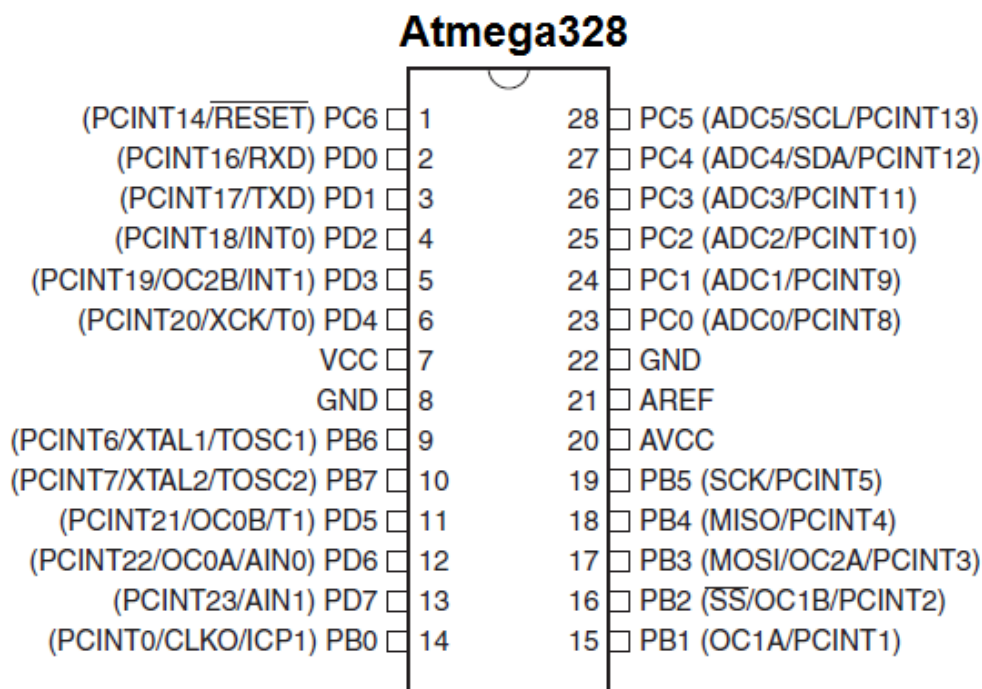


Figura 6.2: Pini microcontroler ATmega328P

Câteva specificații relevante:

- Arhitectura: RISC pe 8 biți, optimizat pentru execuție rapidă și consum redus de energie
- Memoria: 32 KB de memorie internă pentru stocarea codului executabil, 2 KB memorie SRAM pentru date și 1 KB memorie EEPROM
- Convertor Analog-Digital integrat: include un ADC cu rezoluție de 10 biți și 6 canale
- Periferice integrate: 3 Timere/Contoare (2 pe 8 biți, unul pe 16 biți), multiple interfețe de comunicație
- Frecvența de ceas: de obicei 16 MHz
- Porturi I/O: 3

Spre deosebire de arhitectura 8051, ATmega328P elimină necesitatea unui ADC extern, simplificând circuitul și reducând costul total al componentelor și costul de fabricație al cablajului imprimat (PCB), deoarece necesită un număr mai mic de trasee de cupru.

6.3. PIC16F877A

Microcontrolerul PIC16F877A face parte din familia PIC16 și se bazează pe o arhitectură de tip RISC pe 8 biți.

Configurația fizică a pinilor este ilustrată în Figura 6.3 .

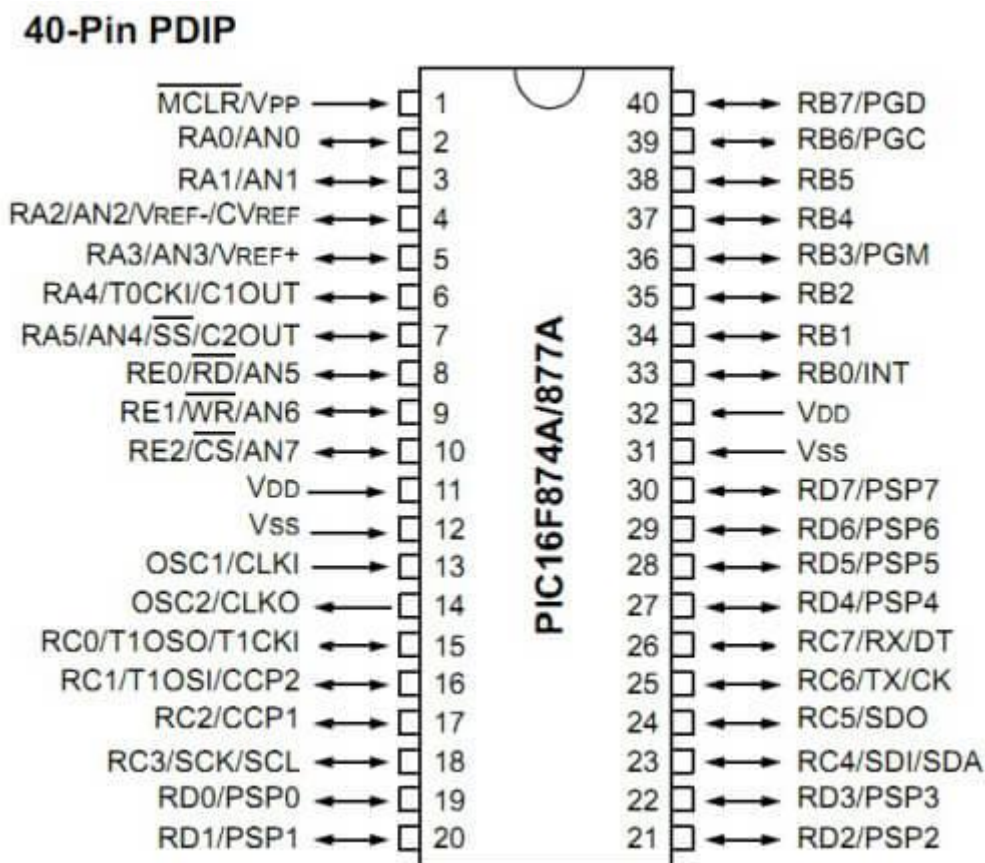


Figura 6.3: Pini microcontroler PIC16F877A

Câteva specificații relevante:

- Arhitectura: RISC pe 8 biți, optimizat pentru execuție rapidă și consum redus de energie
- Memoria: 14 KB de memorie internă pentru stocarea codului executabil, 368 B memorie RAM pentru date și 256 B memorie EEPROM
- Convertor Analog-Digital integrat: include un ADC cu rezoluție de 10 biți și 8 canale
- Periferice integrate: 3 Timere/Contoare (2 pe 8 biți, unul pe 16 biți), multiple interfețe de comunicație
- Frecvența de ceas: de obicei 20 MHz
- Porturi I/O: 5

Spre deosebire de arhitectura 8051, PIC16F877A reprezintă o soluție bună pentru acest proiect, deoarece dispune de modulul ADC intern. Cu toate acestea, comparat cu ATmega328P, PIC16F877A prezintă câteva dezavantaje: memoria internă pentru cod este de peste 2 ori mai mică, iar memoria RAM este mult redusă.

6.4. Concluzia analizei microcontrolerelor

În urma analizei celor trei microprocesoare, ATmega328P este alegerea optimă pentru implementarea acestui proiect, oferind cel mai bun raport între resurse hardware, eficiență de execuție și costuri reduse de implementare.

Nume	Flash	RAM	ADC	Frecvență	Nr. Porturi	Preț
8051	4 KB	128 B	NU	12 MHz	4	2€ - 5€
ATmega328P	32 KB	2 KB	DA	16 MHz	3	2.5€ - 6€
PIC16F877A	14 KB	368 B	DA	20 MHz	5	6€ - 12€

7. Implementarea software a sistemului

7.1. Diagrama bloc de funcționare

Diagrama bloc de funcționare a software-ului dezvoltat pentru microcontroler este prezentată în Figura 7.1. Aceasta prezintă procesul de execuție a programului de la pornire până la bucla infinită de monitorizare a temperaturii. Execuția este structurată în 2 etape principale: etapa de inițializare și bucla principală de funcționare.

După pornirea sistemului, programul începe cu inițializarea porturilor microcontrolerului. Urmează inițializarea modulului LCD, proces în care sunt trimise secvențial comenzile de configurare. Pentru eficientizarea execuției, textul “Temperature:” este afișat o singură dată, înainte de a intra în bucla principală.

Sistemul apoi intră într-un ciclu infinit de achiziție, procesare și afișare.

Se plasează cursorul LCD-ului pe poziția dorită, adică imediat după textul static, astfel la momentul afișării se va suprascrie valoarea veche cu cea nouă.

Microcontrolerul trimite un impuls de comandă convertorului analog-digital pentru a iniția un nou proces de conversie.

Blocul de decizie “Conversie finalizată?” reprezintă soluția pentru sincronizarea microcontrolerului și a ADC-ului. Sistemul implementează o tehnică de polling hardware, monitorizând constant starea pinului de întrerupere. Cât timp conversia este în curs, programul rămâne blocat într-o buclă de așteptare (ramura NU). Când ADC-ul termină conversia va semnaliza acest aspect microcontrolerului, iar execuția continuă (ramura DA).

Odată finalizată conversia, valoarea brută pe 8 biți este citită de pe magistrala de date și trimisă către blocul de “Procesare Valoare”. Aici se aplică un algoritm matematic pentru convertirea valorii brute în grade Celsius și se extrag cifrele individuale (sute, zeci, unități). În final, cifrele procesate sunt convertite în format ASCII și afișate pe ecran, inserându-se și punctul zecimal. Ciclul se încheie cu un “Delay” de câteva sute de milisecunde.

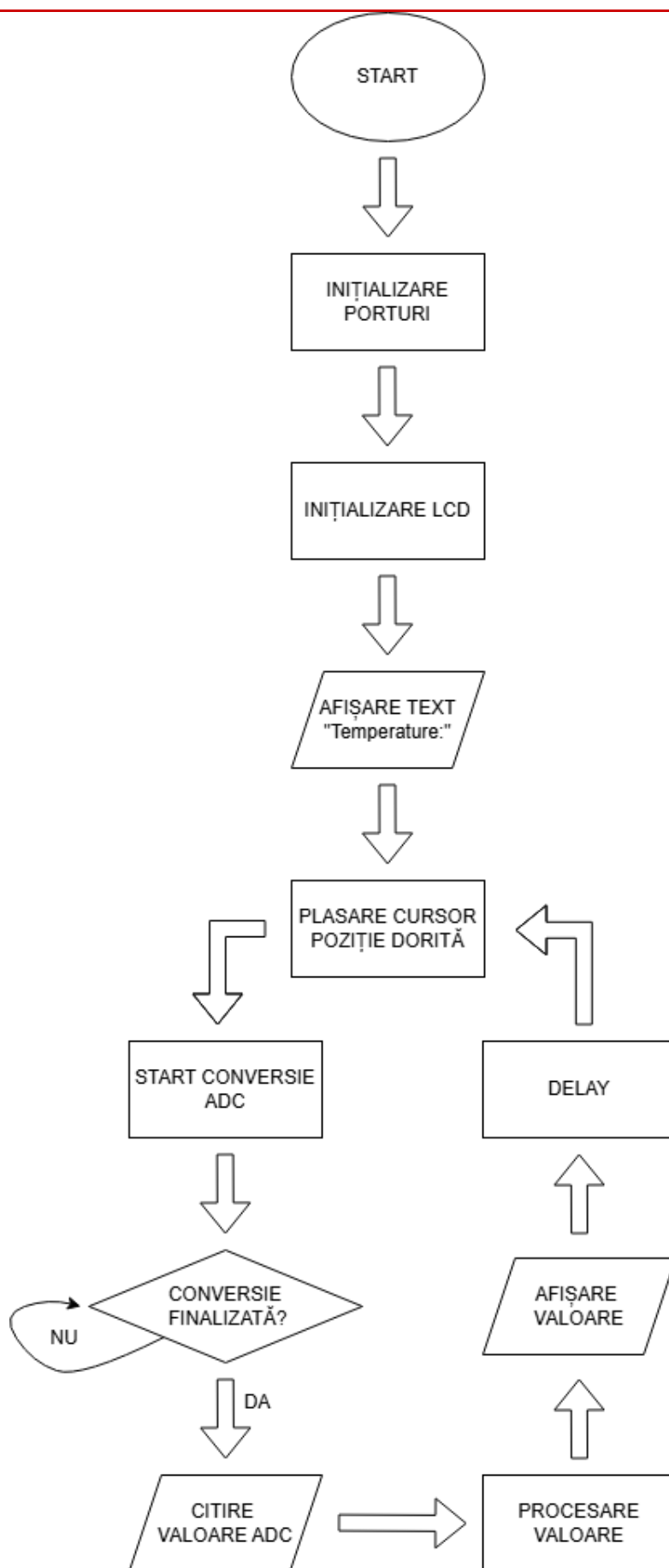


Figura 7.1: Diagrama bloc de funcționare

7.2. Preluarea datelor de la ADC

Prima etapă fundamentală a sistemului o reprezintă achiziția datelor de la convertorul analog-digital. Pentru a asigura o citire corectă, microcontrolerul trebuie să respecte protocolul de comunicare care implică trei pași consecutivi: inițierea conversiei, așteptarea finalizării acesteia și preluarea datelor de pe magistrală.

Deoarece timpul de conversie al ADC0804 este mult mai mare decât timpul de execuție a unei instrucțiuni în microcontroler, sistemul implementează o tehnică de tip polling hardware. În loc să folosească o întârziere fixă, microcontrolerul monitorizează activ starea pinului de întrerupere.

Secvența instrucțiunilor este următoarea:

1. Se trimite un impuls LOW-HIGH pe pinul WR pentru a comanda începerea conversiei.
2. Se așteaptă într-o buclă cât timp pinul INTR este în starea HIGH.
3. Când INTR devine LOW, conversia este gata. Microcontrolerul transmite pe pinul RD un semnal LOW pentru a activa ieșirile ADC-ului, apoi citește valoarea brută (0 - 255) de pe Portul 1.

Funcția `read_adc()` implementează protocolul descris mai sus. În tabelul de mai jos sunt prezentate implementările funcției în limbajele C și Assembly.

<pre> unsigned char read_adc(){ unsigned char val; WR_ADC = 0; WR_ADC = 1; while(INTR == 1); RD_ADC = 0; val = ADC_DATA; RD_ADC = 1; return val; } </pre>	<pre> READ_ADC: CLR WR_ADC NOP NOP SETB WR_ADC WAIT: JB INTR, WAIT CLR RD_ADC NOP NOP MOV A, ADC_DATA SETB RD_ADC RET </pre>
---	--

7.3. Interpretarea datelor

Etapă de procesare are rolul de a transforma o valoare digitală abstractă într-o mărime fizică lizibilă. Această etapă necesită o atenție sporită la calcule, deoarece microcontrolerul este limitat la a lucra cu date de maxim 16 biți.

Valoarea brută preluată de la ADC este un număr întreg pe 8 biți, cuprins între 0 și 255. Sistemul este proiectat să măsoare temperaturi în intervalul 0°C - 40°C. Pentru a obține o precizie de o zecimală lucrând exclusiv cu numere întregi, valoarea finală este calculată direct în zecimi de grad, intervalul fiind 0 – 400.

Ecuția inițială de mapare a intervalelor este:

$$T_{zecimi} = \frac{ADC * 400}{255}$$

Dar pentru a preveni depășirea capacității, adică overflow, la înmulțire și pentru a eficientiza împărțirea, fracția a fost simplificată matematic prin împărțirea la 5, rezultând ecuația care este implementată în cod:

$$T_{zecimi} = \frac{ADC * 80}{51}$$

În limbaj C, utilizarea tipului de date ‘unsigned int’, tip de date pe 16 biți, pentru rezultatul operațiilor previne automat erorile de tip overflow, iar extragerea cifrelor se realizează prin operatorii de modul ‘%’ și împărțire ‘/’.

În schimb, implementarea în limbaj Assembly expune o provocare tehnică: 8051 este un procesor pe 8 biți, astfel rezultatul înmulțirii, $ADC * 80$, va genera un rezultat pe 16 biți, stocat în doi regiștri separați: High Byte în registrul B și Low Byte în registrul A.

Împărțirea unui număr pe 16 biți la valoarea 51 necesită un algoritm de împărțire prin scăderi repetate, ținând cont de împrumutul (Carry) dintre octetul inferior și cel superior:

1. Se scade valoarea 51 din octetul inferior.
2. Se scade 0 împreună cu flag-ul de Carry din octetul superior.
3. Se contorizează de câte ori s-a putut realiza scăderea fără ca numărul pe 16 biți să devină negativ, rezultatul reprezentând temperatura în zecimi de grad

Odată obținut rezultatul, acesta este descompus:

- zecimile de grad (sutele matematice) se extrag prin scăderi repetate ale valorii 100;
- pentru unități și zecimale, restul rămas, fiind garantat mai mic decât 100, încapă într-un singur registru de 8 biți. Folosind instrucțiunea “DIV AB” pentru împărțirea la 10, se obține simultan câtul (unitățile) și restul (zecimale)

Tabelul de mai jos prezintă implementarea conversiei matematice și a divizării pe 16 biți în cele două limbaje:

temp_value = ((unsigned int)val * 80) / 51;	<pre> MOV B, #50h MUL AB MOV R0, B MOV R1, A MOV R2, #00h MOV R3, #00h DIV_51: CLR C MOV A, R1 SUBB A, #33h MOV R1, A MOV A, R0 SUBB A, #00h MOV R0, A JC DIV_DONE INC R3 MOV A, R3 JNZ DIV_51 INC R2 JMP DIV_51 DIV_DONE: MOV TEMP_VAL_H, R2 MOV R0, TEMP_VAL_H MOV TEMP_VAL_L, R3 MOV R1, TEMP_VAL_L </pre>
--	---

7.4. Afișarea datelor pe LCD

Ultima etapă a sistemului constă în transmiterea datelor procesate către modulul LCD. Deoarece ecranul este un modul alfanumeric bazat pe standardul de codare ASCII, valorile brute obținute în etapa de procesare trebuie formate corespunzător înainte de a fi transmise pe magistrala de date.

Cifrele izolate anterior au valori numerice reale cuprinse între 0 și 9. Pentru ca driverul HD44780 al LCD-ului să afișeze corect aceste cifre, trebuie adunată valoarea hexazecimală 0x30, echivalentul caracterului '0'.

Afișarea temperaturii cu o precizie de o zecimală necesită inserarea caracterului punct, '.' cod ASCII 0x2E, între cifra unităților și a zecimalelor.

Pentru a îmbunătăți claritatea afișajului, s-a implementat o condiție astfel încât, dacă temperatura este mai mică de 10 grade, microcontrolerul transmite un spațiu, cod ASCII 0x20, în loc de cifra zero. Astfel sistemul afișează valoarea " 5.2" în loc de "05.2".

Tabelul de mai jos prezintă implementarea codului în cele două limbaje de programare.

<pre> if (tens > 0){ show_char(tens + '0'); } else{ show_char(' '); } show_char(ones + '0'); show_char('.'); show_char(tenth + '0'); </pre>	<pre> PRINT_TEMP: MOV A, R4 JZ SPACE ADD A, #30h JMP PRINT SPACE: MOV A, #20h PRINT: ACALL SHOW_CHAR MOV A, R5 ADD A, #30h ACALL SHOW_CHAR MOV A, #2Eh ACALL SHOW_CHAR MOV A, R6 ADD A, #30h ACALL SHOW_CHAR </pre>
---	--

7.5. Simulări

Validarea funcționalității hardware și software a sistemului a fost realizată utilizând mediul de simulare Proteus. Acest mediu a permis simularea senzorului de temperatură cu valori precise și observarea comportamentului întregului lanț de achiziție, de la variația tensiunii analogice până la formatarea cifrelor de pe ecranul LCD.

Pentru a testa acuratețea afișajului, s-a efectuat o baleiere a temperaturii senzorului pe un interval restrâns, de la 16.0°C la 17.0°C cu un pas constant de 0.2°C. Scopul acestui test a fost verificarea liniarității răspunsului și identificarea eventualelor erori.

Rezultatele obținute în urma simulării sunt ilustrate în Figurile 7.2 – 7.7 și centralizate în tabelul următor:

Temperatură Senzor [°C]	Temperatură Afișată pe LCD	Eroare [°C]
16.0	16.0	0.0
16.2	16.1	- 0.1
16.4	16.4	0.0
16.6	16.6	0.0
16.8	16.7	- 0.1
17.0	17.0	0.0

Se observă o abatere de la valoarea temperaturii senzorului de $\pm 0.1^\circ\text{C}$. Această eroare este o limitare fizică strictă impusă de rezoluția convertorului analog-digital.

Sistemul utilizează un ADC0804, care are o arhitectură pe 8 biți. Astfel el poate genera doar 256 valori digitale distincte. Deoarece sistemul a fost proiectat să măsoare temperaturi de la 0°C la 40°C pe toată scala ADC-ului, rezoluția se calculează astfel:

$$\text{Rezoluție}_{ADC} = \frac{\text{Temperatura}_{MAX}}{\text{Nr. Trepte}} = \frac{40}{255} \approx 0.156^\circ\text{C}/\text{pas}$$

Această valoare demonstrează tehnic sursa erorii: microcontrolerul încearcă să afișeze temperatura cu un pas de 0.1°C, dar ADC-ul furnizează date cu un pas de 0.156°C.

S-au mai realizat două simulări pentru a testa valorile din extremitățile intervalului de temperatură. Rezultatele sunt ilustrate în Figurile 7.8 și 7.9.

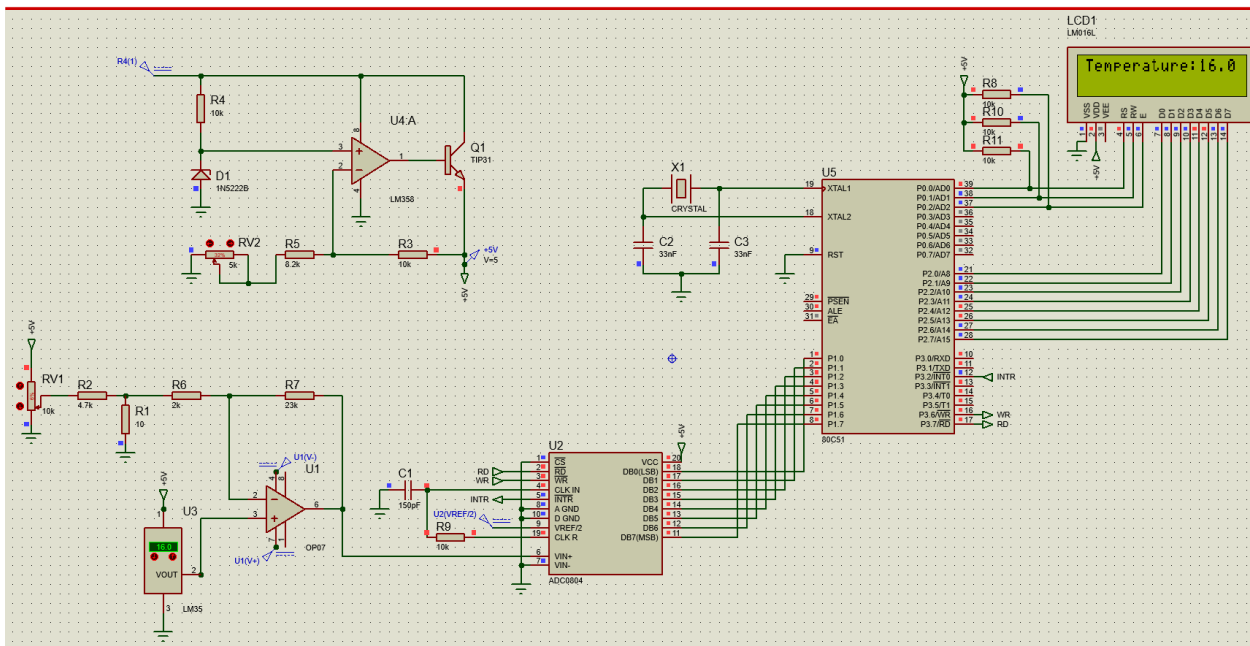


Figura 7.2: Simulare circuit – 16.0°C

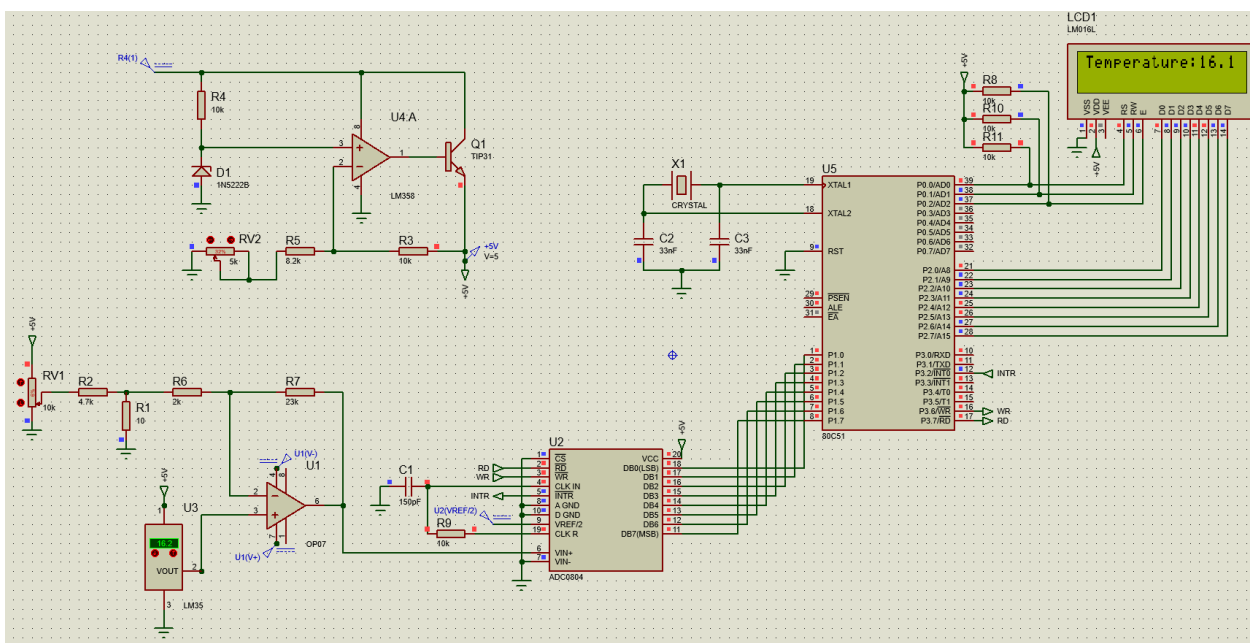


Figura 7.3: Simulare circuit – 16.2°C

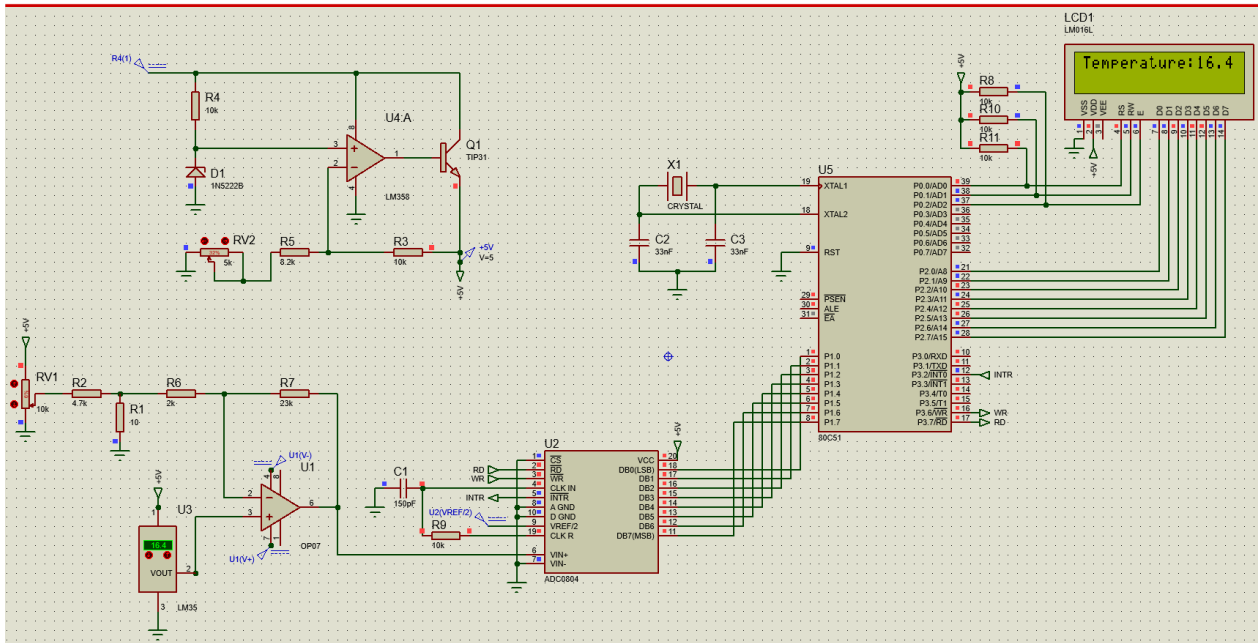


Figura 7.4: Simulare circuit – 16.4°C

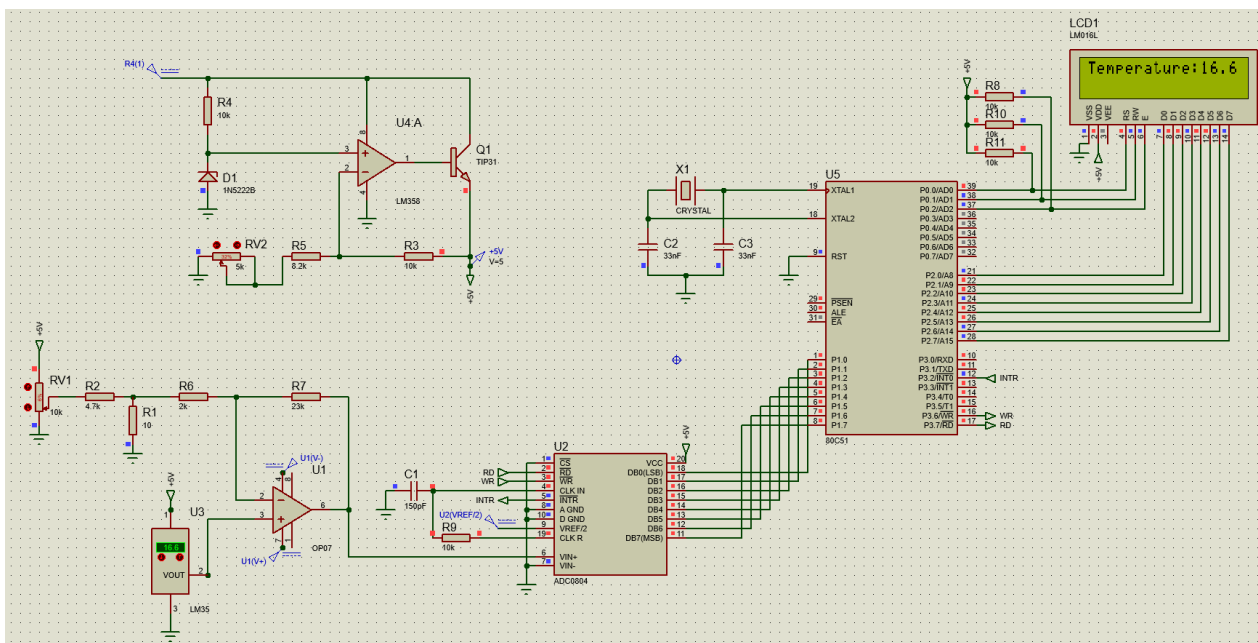


Figura 7.5: Simulare circuit – 16.6°C

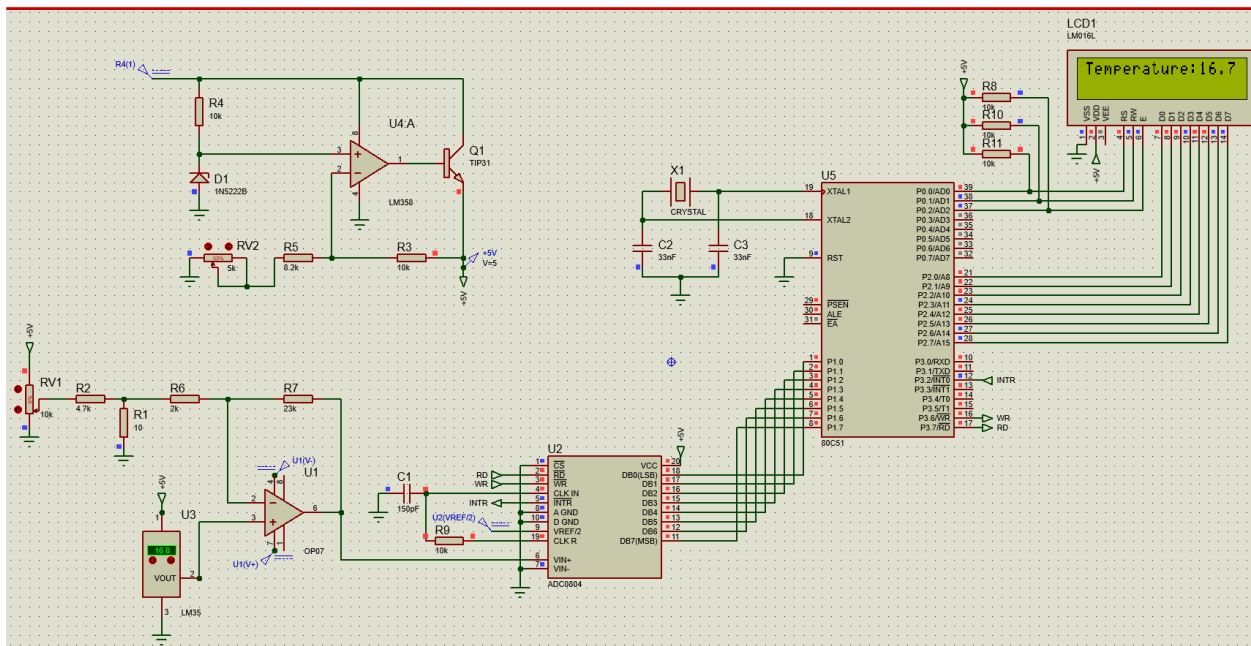


Figura 7.6: Simulare circuit – 16.8°C

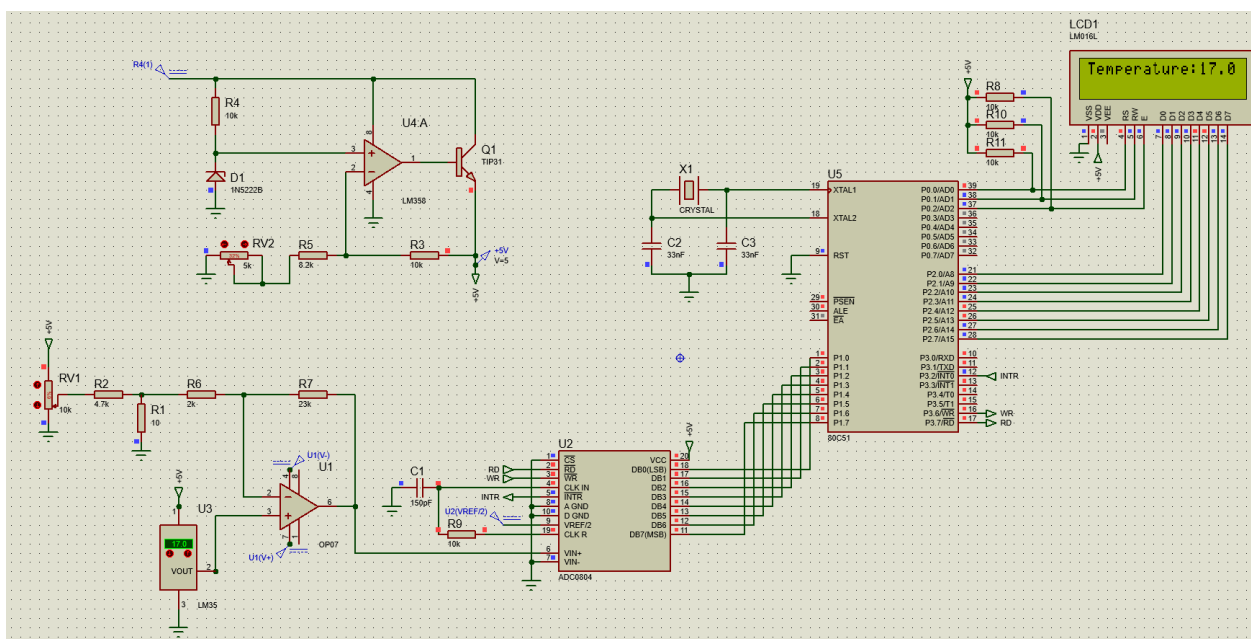


Figura 7.7: Simulare circuit – 17.0°C

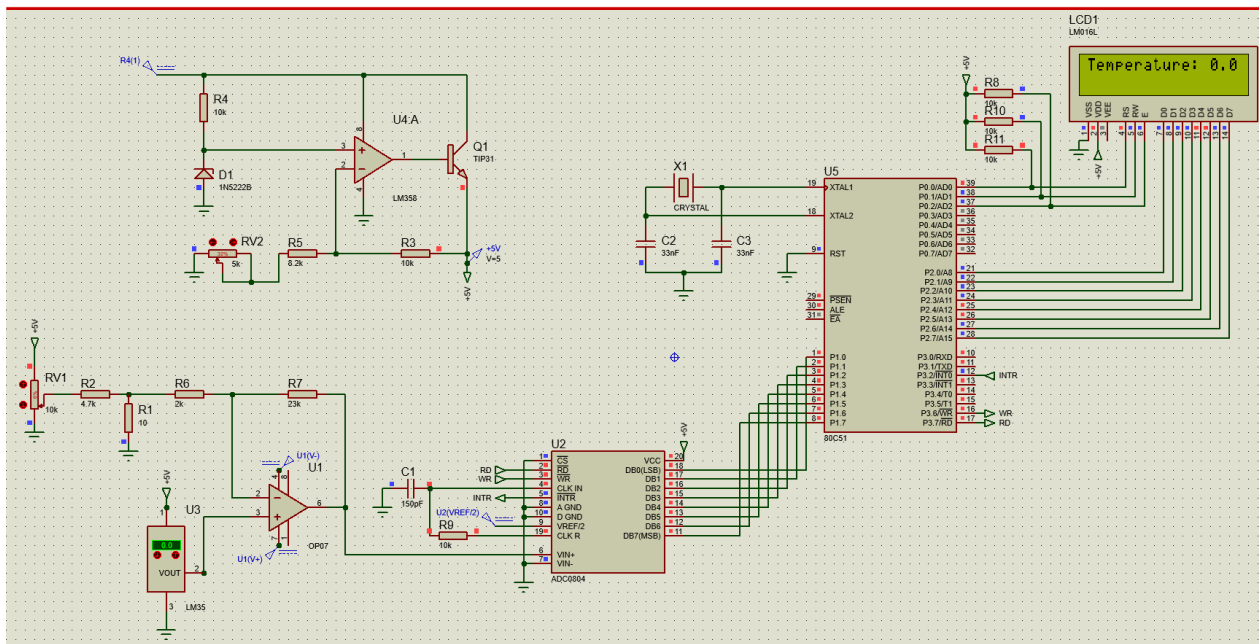


Figura 7.8: Simulare circuit - 0°C

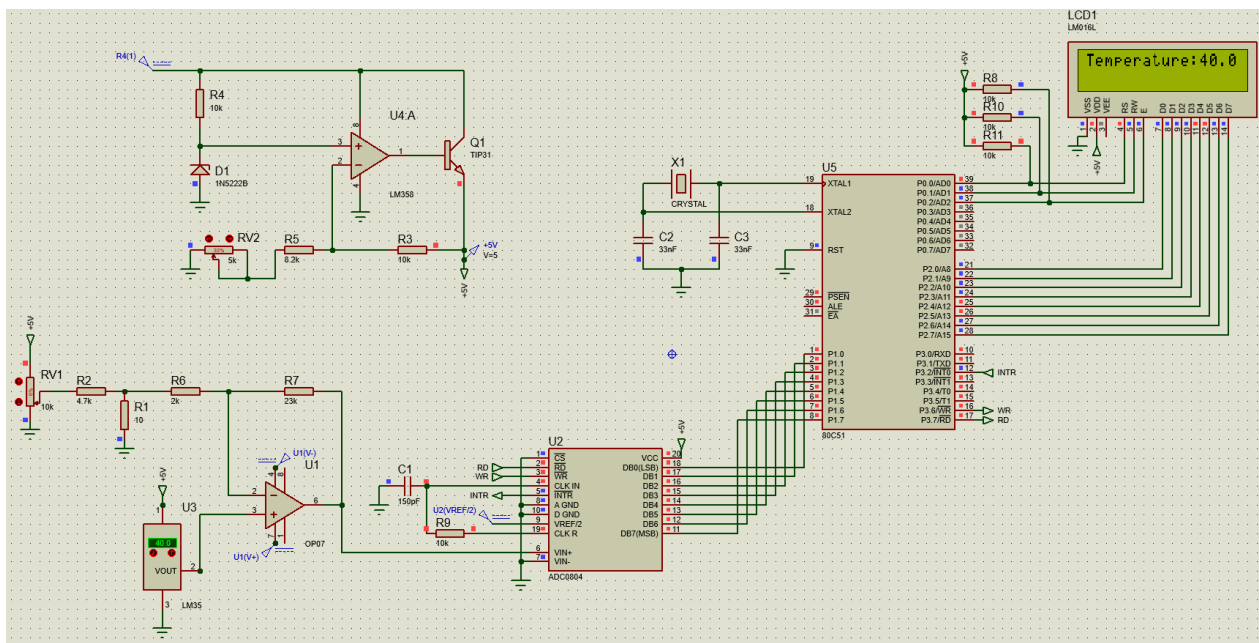


Figura 7.9: Simulare circuit - 40°C

8. Implementarea sistemului de comandă

8.1. Circuitul de comandă

Având în vedere faptul că, pentru acționarea unui releu standard este necesar un curent de aproximativ 60-80 mA, comanda releului electromagnetic din circuit nu se poate realiza direct din portul microcontrolerului 8051, deoarece capacitatea în curent a acestuia este cu mult depășită. Prin urmare, a fost necesară proiectarea unui etaj intermediar de putere, bazat pe un tranzistor NPN, configurat ca întrerupător comandat pe partea de masă (Low-Side Switch).

Implementarea circuitului este ilustrată în Figura 8.1. Așa cum se poate observa în schemă, pentru a asigura o comutație sigură și pentru a proteja microcontrolerul, circuitul integrează o serie de componente active și pasive:

- **Tranzistorul de comutație - Q2:** acționează ca un întrerupător electronic. Când pinul microcontrolerului furnizează un semnal logic HIGH, tranzistorul intră în regiunea de saturație, permițând trecerea curentului prin bobina releului către masă și închizând astfel contactul mecanic.
- **Rezistența de bază - R13:** are rolul de a limita curentul extras din pinul microcontrolerului. Curentul de bază injectat în tranzistor se calculează aplicând legea lui Ohm, considerând tensiunea de ieșire a portului 5V și căderea pe joncțiunea bază-emitor 0.7V.

$$I_B = \frac{5V - 0.7V}{1000\Omega} = 4.3mA$$

Această valoare este sigură pentru portul microcontrolerului și suficientă pentru a asigura saturația tranzistorului Q2.

- **Dioda de protecție - D3:** cunoscută sub denumirea de diodă de regim liber, aceasta este conectată antiparalel cu bobina releului. Deoarece bobina reprezintă o sarcină puternic inductivă, în momentul trecerii tranzistorului Q2 în starea de blocare, câmpul magnetic colapsează brusc, generând un vârf de tensiune autoindusă extrem de mare. Dioda D3 oferă o cale sigură de descărcare a acestui curent tranzitoriu, prevenind distrugerea circuitului.
- **Circuitul de sarcină simulată – LED D2 și R12:** pentru a valida vizual închiderea contactului al releului, a fost implementat un circuit de semnalizare. La cuplarea releului, tensiunea de 5V alimentează un LED roșu. Rezistența R12 limitează curentul prin LED la o valoare de aproximativ 10mA, având în vedere căderea de tensiune tipică pe un LED de această culoare este de 2V.

$$I_{LED} = \frac{5V - 2V}{300\Omega} = 10mA$$

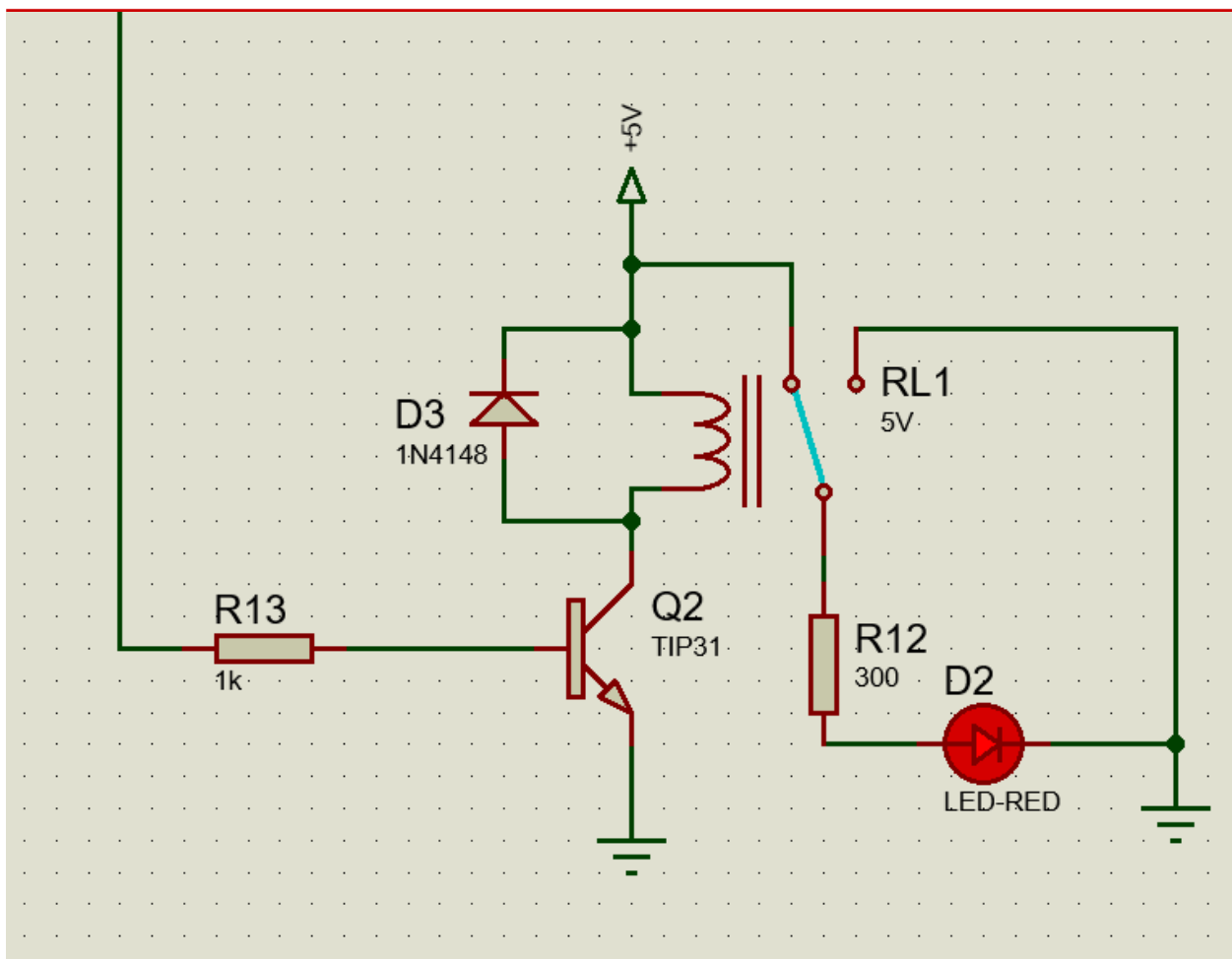


Figura 8.1: Circuitul de comandă

8.2. Logica de comandă a butoanelor

Pentru a permite utilizatorului ajustarea temperaturii de referință, au fost implementate două butoane 'Push-Buttons'. Implementarea hardware a acestora este prezentată în Figura 8.2.

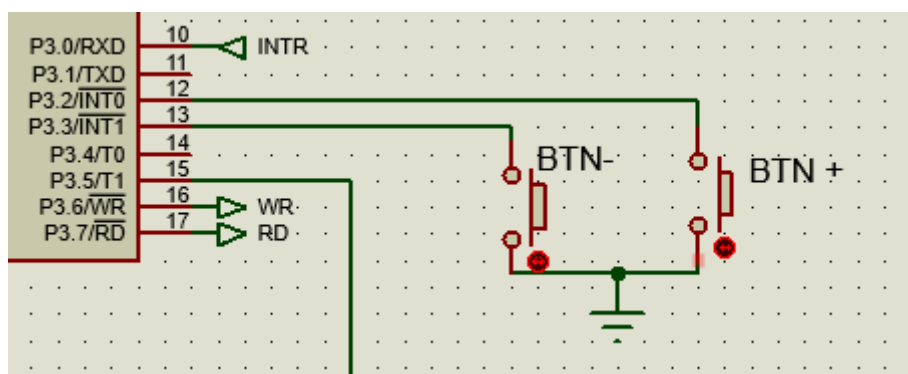


Figura 8.2: Implementarea butoanelor

8.2.1. Conexiunea hardware

Butoanele sunt conectate la pinii *P3.2* și *P3.3* ai microcontrolerului 8051, deoarece acești pini corespund intrărilor pentru întreruperile externe hardware INT0 și INT1.

Din punct de vedere electric, butoanele sunt configurate în modul *Active-Low*. Pinii portului 3 dispun de rezistențe interne de pull-up, menținând un nivel logic HIGH în starea de repaus. La apăsarea butonului, circuitul se închide la masă, aducând pinul în starea logică LOW și declanșând rutina de tratare a întreruperii.

8.2.2. Implementarea Software

Din cauza prezenței impulsurilor parazite la apăsarea sau eliberarea butonului, a fost necesară implementarea unui algoritm de *Debounce*.

Algoritmul funcționează în felul următor: după declanșarea întreruperii, execuția este suspendată pentru 20 ms, apoi se citește din nou starea pinului pentru a valida că butonul încă este apăsat. După modificarea valorii de referință, algoritmul blochează execuția într-o buclă de așteptare până la eliberarea completă a butonului, urmată de o nouă suspendare a execuției pentru 20ms pentru a filtra zgomotul de decuplare.

Pentru a asigura o funcționare sigură a termostatlui și pentru a preveni setarea unor valori anormale, au fost impuse limite stricte pentru temperatura de referință: un minim de 10.0°C și un maxim de 30.0°C.

Mai jos este prezentată implementarea logicii pentru tratarea întreruperii generate de apăsarea butonului “BTN+”, realizată comparativ în limbajele C și Assembly.

În limbajul C, protecția se realizează simplu, printr-o condiție logică multiplă evaluată direct în interiorul rutinei de tratare a întreruperii externe.

```
void plus() interrupt 0{
    delay(20);
    if(BTN_PLUS == 0 && set_value < max_value)
    {
        set_value += 5;
        update_lcd_flag = 1;
    }
    while(BTN_PLUS == 0);
    delay(20);
}
```

În limbajul Assembly, deoarece se operează strict pe 8 biți, compararea valorii curente pe 16 biți cu limita de 300 se realizează manual, printr-o scădere cu împrumut.

```
ISR_PLUS:
    PUSH PSW
    PUSH ACC
    PUSH B

    MOV A, #14h
    MOV B, #00h
    ACALL DELAY

    JB BTN_PLUS, EXIT_PLUS
```

```

CLR C
MOV A, SET_VAL_L
SUBB A, #2Ch
MOV A, SET_VAL_H
SUBB A, #01h
JNC WAIT_RELEASE_PLUS

MOV A, SET_VAL_L
ADD A, #05h
MOV SET_VAL_L, A
MOV A, SET_VAL_H
ADDC A, #00h
MOV SET_VAL_H, A

SETB FLAG_LCD

WAIT_RELEASE_PLUS: JNB BTN_PLUS, WAIT_RELEASE_PLUS

MOV A, #14h
MOV B, #00h
ACALL DELAY

EXIT_PLUS:
POP B
POP ACC
POP PSW
RETI

```

8.2.3. Gestiunea întreruperilor

Comunicare microcontrolerului cu afișajul LCD presupune respectarea unor diagrame de timp stricte pentru transmiterea comenzilor și a datelor. Pentru a preveni coruperea informației afișate, un exemplu este prezentat în Figura 8.3, pe durata execuției blocurilor de cod responsabile cu afișarea temperaturii, întreruperile globale sunt dezactivate temporar. Această măsură de siguranță garantează că o eventuală apăsare de buton nu va suspenda și desincroniza protocolul de comunicație cu LCD-ul. Imediat după terminarea transmisiei datelor către ecran, întreruperile sunt reactivate, permițând sistemului să răspundă din nou la comenzile utilizatorului.

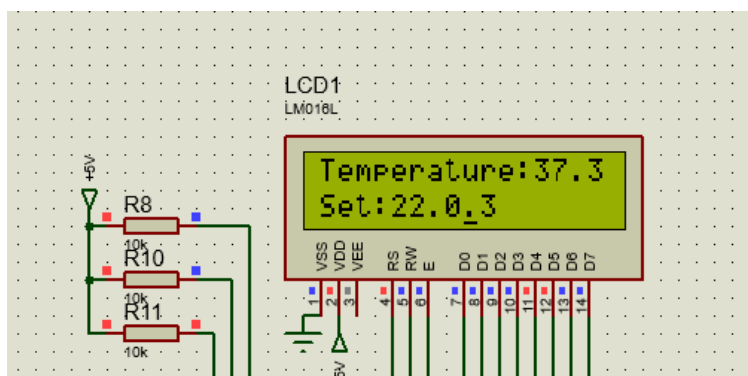


Figura 8.3: Exemplu de corupere a datelor

8.3. Algoritmul de control si implementarea histerezei

Pentru implementarea algoritmului de decizie cu histereză, s-a definit o marjă de $\pm 0.5^{\circ}\text{C}$ în jurul valorii setate, aceasta fiind reprezentată de valoarea matematică 5 în logica microcontrolerului. Astfel, releul va cupla doar când temperatura scade sub limita inferioară și se va decupla doar când temperatura depășește limita superioară. Cât timp temperatura se află în interiorul acestui interval, starea releului rămâne neschimbată, asigurând o funcționare lină a elementului de încălzire.

În C, implementarea histerezei este directă și intuitivă. Funcția 'process_data' primește valoarea de la convertorul analog-digital, o convertește în temperatura reală, iar apoi utilizează o structură decizională simplă de tip 'if-else if' pentru a comanda pinul releului.

```
void process_data(unsigned char val){

    temp_value = ((unsigned int)val * 80 ) / 51;

    if(temp_value < (set_value - 5)){
        RELAY = 1;
    }
    else if( temp_value > (set_value + 5)){
        RELAY = 0;
    }

    show_value(temp_value);
}
```

În Assembly, arhitectura pe 8 biți necesită o abordare mai complexă. Compararea valorii curente cu limitele calculate presupune efectuarea unei operații de scădere pe 16 biți cu împrumut. Dacă operația de scădere generează un Flag de Carry, înseamnă că valoarea curentă este strict mai mică decât limita cu care este comparată. Pentru a nu distruge datele originale de pe ecran, limitele sunt calculate și stocate temporar în regiștrii R4 și R5.

```
PROCESS_DATA:
    MOV B, #50h
    MUL AB
    MOV R0, B
    MOV R1, A

    MOV R2, #00h
    MOV R3, #00h

    DIV_51:
        CLR C

        MOV A, R1
        SUBB A, #33h
        MOV R1, A

        MOV A, R0
```

```
SUBB A, #00h
MOV R0, A

JC DIV_DONE
INC R3
MOV A, R3
JNZ DIV_51
INC R2
JMP DIV_51
DIV_DONE:
MOV TEMP_VAL_H, R2
MOV R0, TEMP_VAL_H
MOV TEMP_VAL_L, R3
MOV R1, TEMP_VAL_L

VERIFY_1:
CLR C
MOV A, SET_VAL_L
SUBB A, #05h
MOV R4, A
MOV A, SET_VAL_H
SUBB A, #00h
MOV R5, A

CLR C
MOV A, TEMP_VAL_L
SUBB A, R4
MOV A, TEMP_VAL_H
SUBB A, R5
JNC VERIFY_2

SETB RELAY
SJMP FINISH
VERIFY_2:
MOV A, SET_VAL_L
ADD A, #05h
MOV R4, A
MOV A, SET_VAL_H
ADDC A, #00h
MOV R5, A

CLR C
MOV A, TEMP_VAL_L
SUBB A, R4
MOV A, TEMP_VAL_H
SUBB A, R5
JC FINISH

CLR RELAY
FINISH:
```

```
ACALL SHOW_VALUE
RET
```

8.4. Simulări

Pentru a demonstra funcționalitatea circuitului implementat, au fost realizate mai multe simulări. În secțiunea următoare sunt prezentate capturi de ecran ce surprind răspunsul sistemului în cele mai importante scenarii de funcționare.

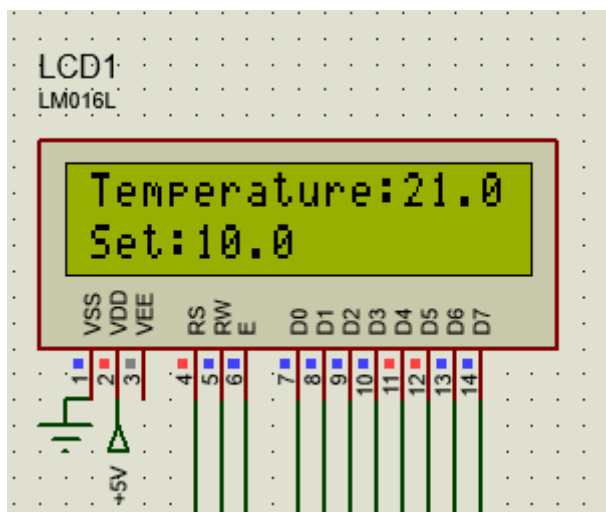


Figura 8.4: Temperatura de referință minimă

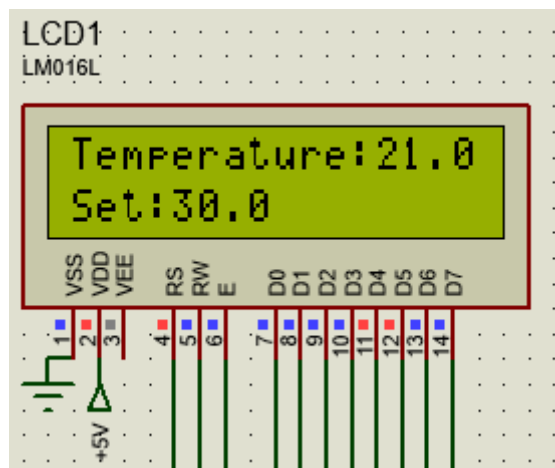


Figura 8.5: Temperatura de referință maximă

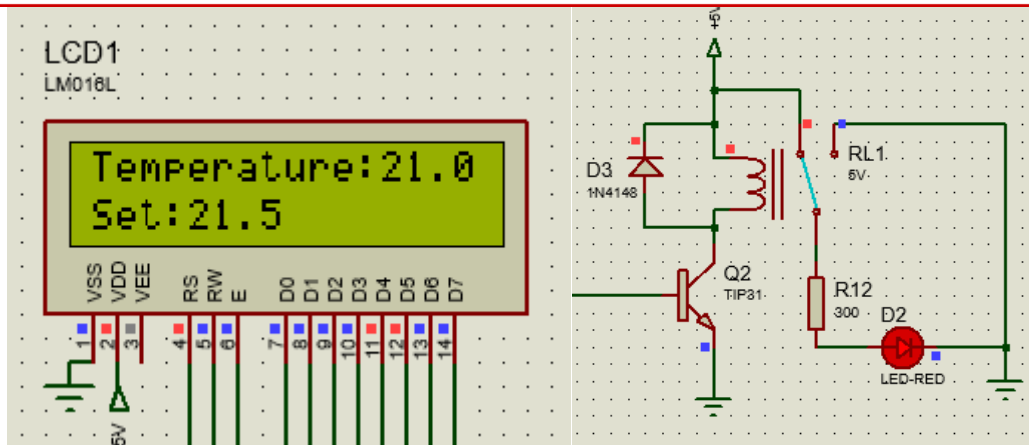


Figura 8.6: Temperatura de start & Răspunsul sistemului

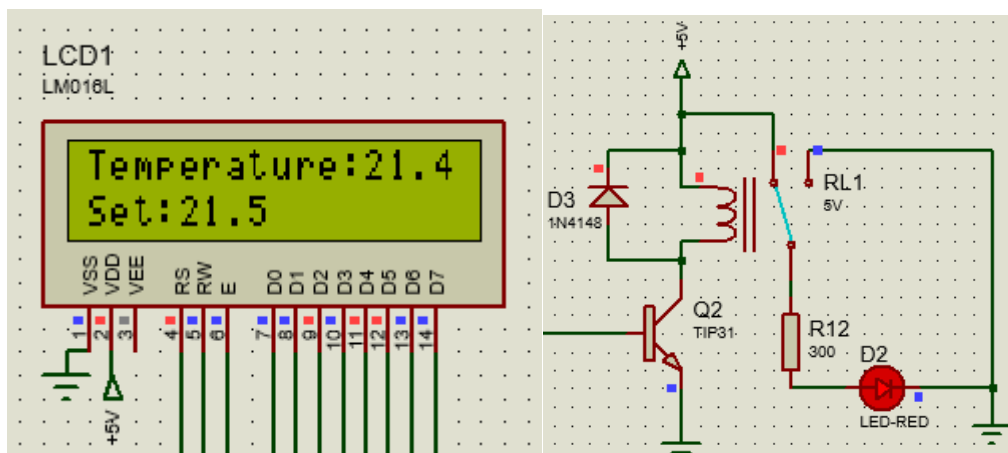


Figura 8.7: Temperatura intermediară & Răspunsul sistemului

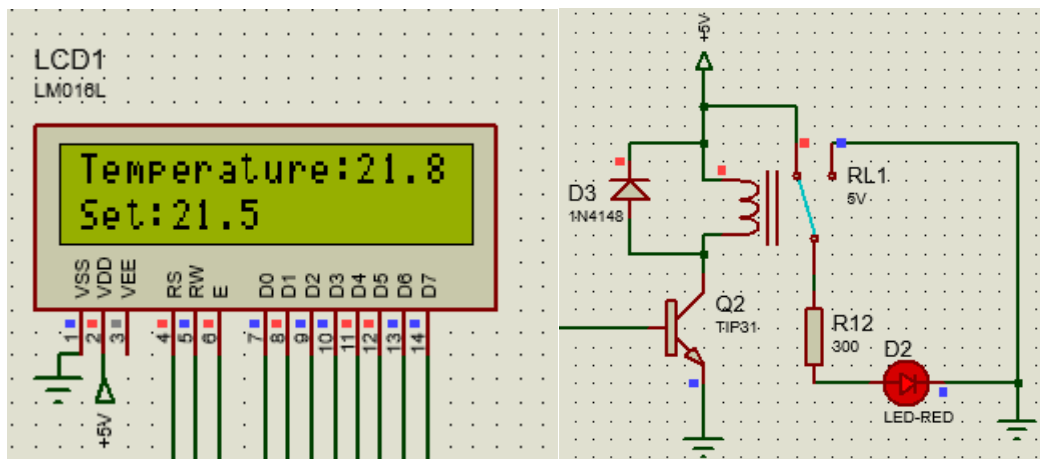


Figura 8.8: Temperatura intermediară & Răspunsul sistemului

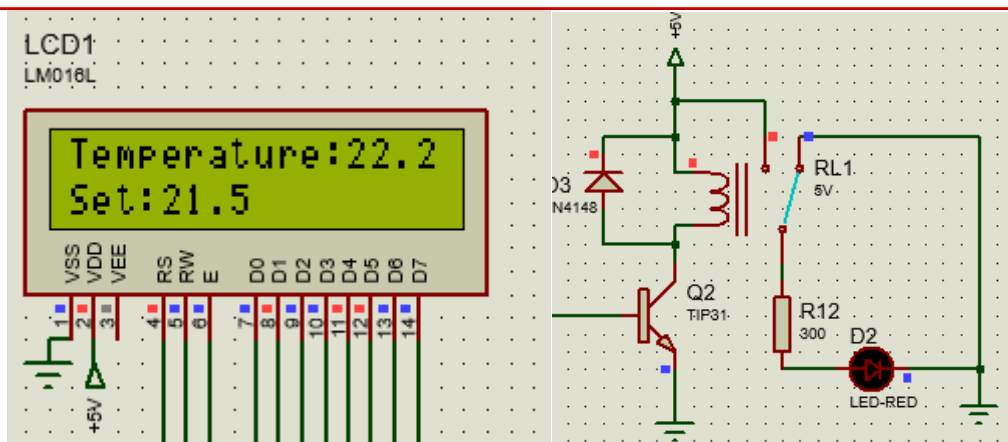


Figura 8.9: Temperatura intermediară & Răspunsul sistemului

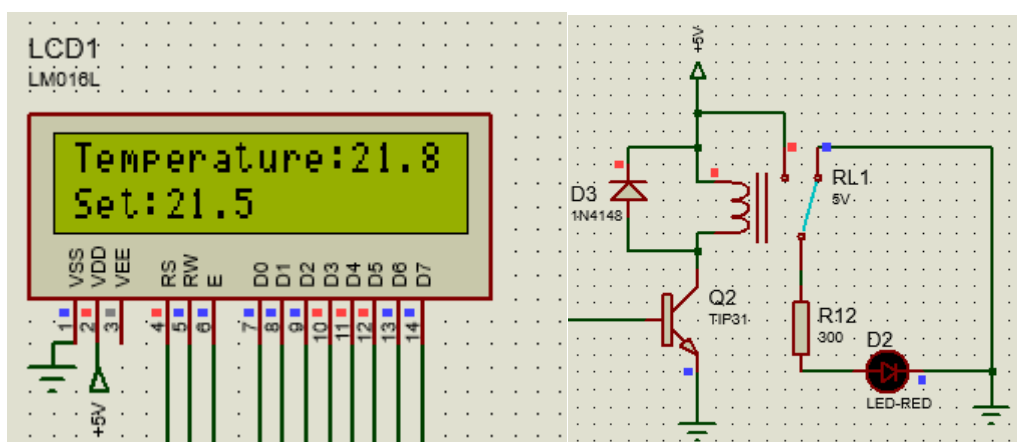


Figura 8.10: Temperatura intermediară & Răspunsul sistemului

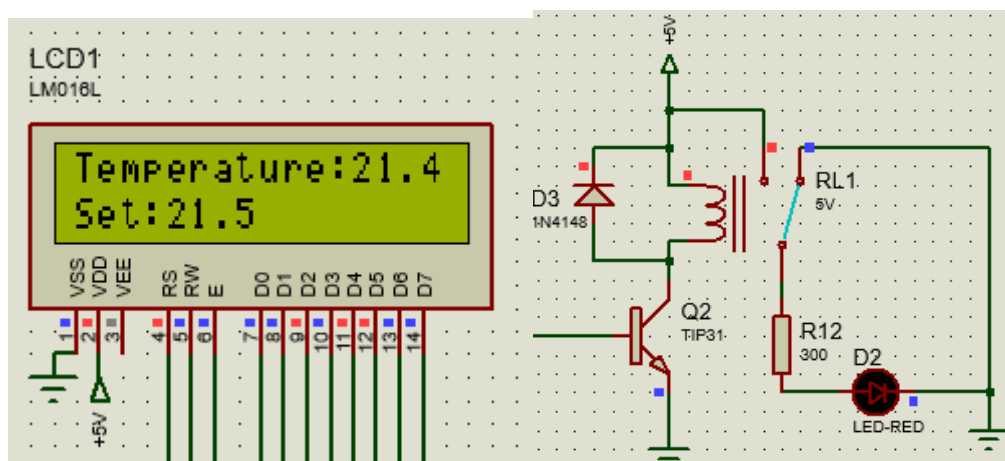


Figura 8.11: Temperatura intermediară & Răspunsul sistemului

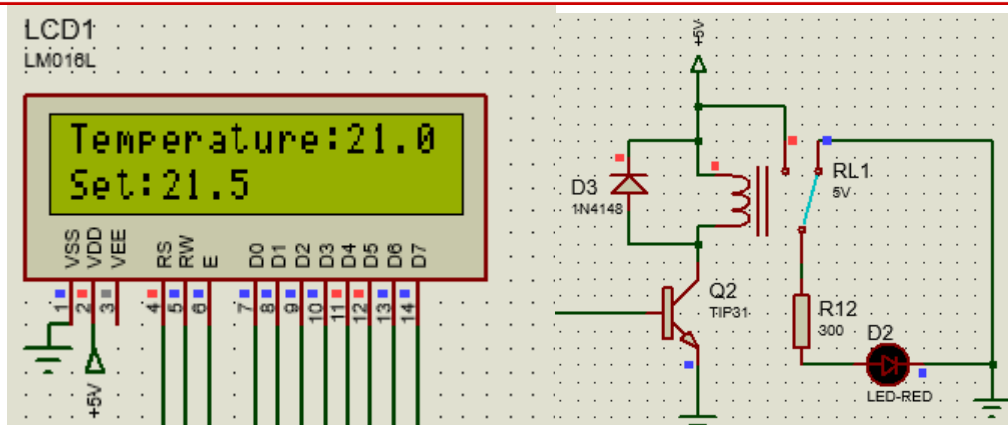


Figura 8.12: Temperatura intermediară & Răspunsul sistemului

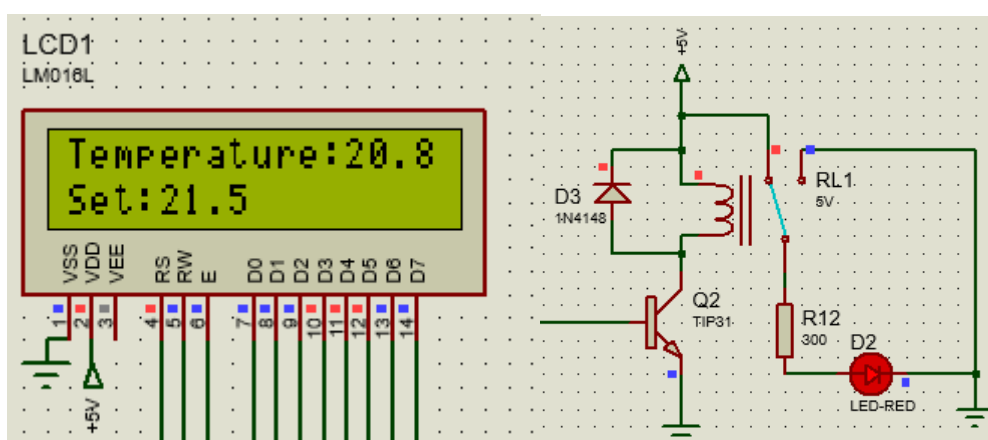


Figura 8.13: Temperatura intermediară & Răspunsul sistemului

9. Cod C & Assembly

9.1. Cod C

```
#include <REG51.H>

#define ADC_DATA P1
#define LCD P2

sbit RS = P0^0;
sbit RW = P0^1;
sbit E = P0^2;
sbit INTR = P3^0;
sbit BTN_PLUS = P3^2;
sbit BTN_MINUS = P3^3;
sbit RELAY = P3^5;
sbit WR_ADC = P3^6;
sbit RD_ADC = P3^7;

unsigned char code string[] = "Temperature:";
unsigned char code set[] = "Set:";
unsigned int set_value = 210;
unsigned int max_value = 300;
unsigned int min_value = 100;
unsigned int temp_value;
bit update_lcd_flag = 0;

void delay(int ms);
void command(int hex);
void setup_lcd();
void show_char(unsigned char x);
void show_value(unsigned int val);
void process_data(unsigned char val);
unsigned char read_adc();

void main(){
    ADC_DATA = 0xFF;
    LCD = 0x00;
    RW = 0;
    RELAY = 0;
    INTR = 1;
    BTN_PLUS = 1;
    BTN_MINUS = 1;
    IT0 = 1;
    IT1 = 1;
    IE = 0x85;
    setup_lcd();

    while(1){
```

```

    EA = 0;
    command(0x8C);
    process_data(read_adc());
    EA = 1;

    if(update_lcd_flag){
        EA = 0;
        command(0xC4);
        show_value(set_value);
        EA = 1;
        update_lcd_flag = 0;
    }

    delay(300);
}

void plus() interrupt 0{
    delay(20);
    if(BTN_PLUS == 0 && set_value < max_value)
    {
        set_value += 5;
        update_lcd_flag = 1;
    }
    while(BTN_PLUS == 0);
    delay(20);
}

void minus() interrupt 2{
    delay(20);
    if(BTN_MINUS == 0 && set_value > min_value){
        set_value -= 5;
        update_lcd_flag = 1;
    }
    while(BTN_MINUS == 0);
    delay(20);
}

void delay(int ms){
    unsigned int j, i;
    for(j=0; j< ms; j++){
        for(i = 0; i< 112; i++){
        }
    }
}

void command(int hex){
    RS = 0;
    LCD = hex;
    E = 1;

```

```
    delay(1);
    E = 0;
}

void setup_lcd(){
    unsigned int i;

    command(0x38);
    delay(2);

    command(0x0C);
    delay(2);

    command(0x01);
    delay(2);

    command(0x06);
    delay(2);

    for(i=0; i < 0x0C; i++){
        show_char(string[i]);
    }

    command(0xC0);
    delay(2);

    for( i=0; i < 0x04; i++){
        show_char(set[i]);
    }
    show_value(set_value);
}

void show_char(unsigned char x){
    RS = 1;
    LCD = x;
    E = 1;
    delay(10);
    E = 0;
}

unsigned char read_adc(){
    unsigned char val;

    WR_ADC = 0;
    WR_ADC = 1;

    while(INTR == 1);

    RD_ADC = 0;
    val = ADC_DATA;
```

```

    RD_ADC = 1;

    return val;
}

void process_data(unsigned char val){

    temp_value = ((unsigned int)val * 80 ) / 51;

    if(temp_value < (set_value - 5)){
        RELAY = 1;
    }
    else if( temp_value > (set_value + 5)){
        RELAY = 0;
    }

    show_value(temp_value);
}

void show_value(unsigned int val){
    unsigned char tens, ones, tenth;

    tens = val / 100;
    ones = (val % 100) / 10;
    tenth = val % 10;

    if (tens > 0){
        show_char(tens + '0');
    }
    else{
        show_char(' ');
    }

    show_char(ones + '0');
    show_char('.');
    show_char(tenth + '0');
}

```

9.2. Cod Assembly

```

del EQU 99h
ADC_DATA EQU P1
LCD EQU P2
RS EQU P0.0
RW EQU P0.1
E EQU P0.2
INTR EQU P3.0
BTN_PLUS EQU P3.2
BTN_MINUS EQU P3.3

```

```
RELAY EQU P3.5
WR_ADC EQU P3.6
RD_ADC EQU P3.7

SET_VAL_L EQU 30h
SET_VAL_H EQU 31h
TEMP_VAL_L EQU 32h
TEMP_VAL_H EQU 33h
FLAG_LCD BIT 20h.0

ORG 0000h
    LJMP MAIN

ORG 0003h
    LJMP ISR_PLUS

ORG 0013h
    LJMP ISR_MINUS

ORG 0050h
    text_temp: DB "Temperature:",0
    text_set: DB "Set:",0

ORG 0100h

MAIN:
    MOV ADC_DATA, #0FFh
    MOV LCD, #00h
    CLR RW
    CLR RELAY
    SETB INTR
    SETB BTN_PLUS
    SETB BTN_MINUS
    SETB IT0
    SETB IT1
    MOV IE, #85h

    MOV SET_VAL_H, #00h
    MOV SET_VAL_L, #0D2h

    ACALL SETUP_LCD

    LOOP:
        CLR EA
        MOV A, #8Ch
        ACALL COMMAND
        ACALL READ_ADC
        ACALL PROCESS_DATA
        SETB EA
```



```
JNB FLAG_LCD, NOT_FLAG
CLR EA
MOV A, #0C4h
ACALL COMMAND

MOV R2, SET_VAL_H
MOV R3, SET_VAL_L

ACALL SHOW_VALUE
SETB EA
CLR FLAG_LCD
NOT_FLAG:
MOV A, #2Ch
MOV B, #01h
ACALL DELAY
JMP LOOP
```

ISR_PLUS:

```
PUSH PSW
PUSH ACC
PUSH B
```

```
MOV A, #14h
MOV B, #00h
ACALL DELAY
```

```
JB BTN_PLUS, EXIT_PLUS
```

```
CLR C
MOV A, SET_VAL_L
SUBB A, #2Ch
MOV A, SET_VAL_H
SUBB A, #01h
JNC WAIT_RELEASE_PLUS
```

```
MOV A, SET_VAL_L
ADD A, #05h
MOV SET_VAL_L, A
MOV A, SET_VAL_H
ADDC A, #00h
MOV SET_VAL_H, A
```

```
SETB FLAG_LCD
```

```
WAIT_RELEASE_PLUS: JNB BTN_PLUS, WAIT_RELEASE_PLUS
```

```
MOV A, #14h
MOV B, #00h
ACALL DELAY
```

```
EXIT_PLUS:
POP B
POP ACC
POP PSW
RETI

ISR_MINUS:
PUSH PSW
PUSH ACC
PUSH B

MOV A, #14h
MOV B, #00h
ACALL DELAY

JB BTN_MINUS, EXIT_MINUS

CLR C
MOV A, SET_VAL_L
SUBB A, #65h
MOV A, SET_VAL_H
SUBB A, #00h
JC WAIT_RELEASE_MINUS

CLR C
MOV A, SET_VAL_L
SUBB A, #05h
MOV SET_VAL_L, A
MOV A, SET_VAL_H
SUBB A, #00h
MOV SET_VAL_H, A

SETB FLAG_LCD

WAIT_RELEASE_MINUS: JNB BTN_MINUS, WAIT_RELEASE_MINUS

MOV A, #14h
MOV B, #00h
ACALL DELAY

EXIT_MINUS:
POP B
POP ACC
POP PSW
RETI

DELAY:
PUSH 07
PUSH ACC
ORL A, B
```

JNZ OK

POP ACC

POP 07

RET

OK:

POP ACC

DEC_LSB:

ACALL ONEMIL

DJNZ ACC, DEC_LSB

CJNE A, B, DEC_MSB

POP 07

RET

DEC_MSB:

DEC B

SJMP DEC_LSB

ONEMIL:

MOV R7, #del

TIMER:

NOP

NOP

NOP

NOP

DJNZ R7, TIMER

NOP

RET

SETUP_LCD:

MOV A, #38h

ACALL COMMAND

MOV A, #0Ch

ACALL COMMAND

MOV A, #01h

ACALL COMMAND

MOV A, #06h

ACALL COMMAND

MOV DPTR, #text_temp

SHOW_TEXT_1:

CLR A

MOVC A, @A+DPTR

JZ END_PRINT_1

ACALL SHOW_CHAR

```

        INC DPTR
        JMP SHOW_TEXT_1
END_PRINT_1:

MOV A, #0C0h
ACALL COMMAND

MOV DPTR, #text_set
SHOW_TEXT_2:
    CLR A
    MOVC A, @A+DPTR
    JZ END_PRINT_2
    ACALL SHOW_CHAR
    INC DPTR
    JMP SHOW_TEXT_2
END_PRINT_2:
    MOV R2, SET_VAL_H
    MOV R3, SET_VAL_L
    ACALL SHOW_VALUE
RET

COMMAND:
    CLR RS
    MOV LCD, A
    SETB E
    ACALL ONEMIL
    ACALL ONEMIL
    CLR E
RET

SHOW_CHAR:
    SETB RS
    MOV LCD, A
    SETB E
    MOV A, #0Ah
    MOV B, #00h
    ACALL DELAY
    CLR E
RET

READ_ADC:
    CLR WR_ADC
    NOP
    NOP
    SETB WR_ADC

WAIT: JB INTR, WAIT
    CLR RD_ADC
    NOP
    NOP

```

```
MOV A, ADC_DATA
SETB RD_ADC
RET

PROCESS_DATA:
MOV B, #50h
MUL AB
MOV R0, B
MOV R1, A

MOV R2, #00h
MOV R3, #00h

DIV_51:
CLR C

MOV A, R1
SUBB A, #33h
MOV R1, A

MOV A, R0
SUBB A, #00h
MOV R0, A

JC DIV_DONE
INC R3
MOV A, R3
JNZ DIV_51
INC R2
JMP DIV_51
DIV_DONE:
MOV TEMP_VAL_H, R2
MOV R0, TEMP_VAL_H
MOV TEMP_VAL_L, R3
MOV R1, TEMP_VAL_L

VERIFY_1:
CLR C
MOV A, SET_VAL_L
SUBB A, #05h
MOV R4, A
MOV A, SET_VAL_H
SUBB A, #00h
MOV R5, A

CLR C
MOV A, TEMP_VAL_L
SUBB A, R4
MOV A, TEMP_VAL_H
SUBB A, R5
```

```
JNC VERIFY_2

SETB RELAY
SJMP FINISH
VERIFY_2:
MOV A, SET_VAL_L
ADD A, #05h
MOV R4, A
MOV A, SET_VAL_H
ADDC A, #00h
MOV R5, A

CLR C
MOV A, TEMP_VAL_L
SUBB A, R4
MOV A, TEMP_VAL_H
SUBB A, R5
JC FINISH

CLR RELAY
FINISH:
ACALL SHOW_VALUE
RET

SHOW_VALUE:
MOV R4, #00h
MOV R5, #00h
MOV R6, #00h

LOOP_TENS:
CLR C

MOV A, R3
SUBB A, #64h
MOV R3, A

MOV A, R2
SUBB A, #00h
MOV R2, A

JC LOOP_TENS_DONE
INC R4
JMP LOOP_TENS
LOOP_TENS_DONE:
MOV A, R3
ADD A, #64h
MOV R3, A

ONES_AND_TENTH:
MOV A, R3
```

```
MOV B, #0Ah
DIV AB
MOV R5, A
MOV R6, B
PRINT_TEMP:
MOV A, R4
JZ SPACE
ADD A, #30h
JMP PRINT
SPACE:
MOV A, #20h
PRINT:
ACALL SHOW_CHAR

MOV A, R5
ADD A, #30h
ACALL SHOW_CHAR

MOV A, #2Eh
ACALL SHOW_CHAR

MOV A, R6
ADD A, #30h
ACALL SHOW_CHAR
RET
END
```

10. Concluzii

În concluzie, obiectivul principal al proiectului a fost îndeplinit cu succes prin realizarea unui termostat digital funcțional, validat în mediul de simulare Proteus. Sistemul este capabil să achiziționeze date de la senzorul analogic LM35, să prelucreze informația la nivel software evitând depășirile de memorie și să comande un etaj de putere cu releu, utilizând o marjă de histereză stabilă. Implementarea duală a codului, atât în limbajul C, cât și în limbajul de asamblare, a demonstrat o înțelegere aprofundată a arhitecturii hardware a microcontrolerului 8051 și a modului în care resursele acestuia pot fi gestionate la nivel inferior.

11. Bibliografie

- [1] Hitachi, „LM016L Dot Matrix Liquid Crystal Display Module”, Datasheet, https://digsys.upc.edu/csd/chips/classic/LM016L_Hitachi.pdf .
- [2] Innovation of Engineers, „LCD 16×2 (LM016L)”, *ECE Study Centre*, <https://embeddedcenter.wordpress.com/ece-study-centre/display-module/lcd-16x2-lm016l/> .
- [3] Microchip Technology Inc., „Atmega328P Automotive Microcontrollers Datasheet”, Datasheet, https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.
- [4] Microchip Technology Inc., „PIC16F87XA Data Sheet”, Datasheet, <https://ww1.microchip.com/downloads/en/devicedoc/39582b.pdf>.
- [5] Texas Instruments, „LM358 Dual Operational Amplifiers Datasheet”, Datasheet, <https://www.ti.com/lit/ds/symlink/lm358.pdf>.
- [6] Texas Instruments, „Basic Calculation of a Buck Converter's Power Stage”, notă de aplicație, <https://www.ti.com/lit/an/slva477b/slva477b.pdf>.
- [7] ON Semiconductor, „TIP31A (NPN) Complementary Silicon Plastic Power Transistors”, Datasheet, <https://www.onsemi.com/download/data-sheet/pdf/tip31a-d.pdf>.
- [8] ON Semiconductor, „TIP32C (PNP) Complementary Silicon Plastic Power Transistors”, Datasheet, <https://www.onsemi.com/pdf/datasheet/tip32c-d.pdf>.
- [9] Taiwan Semiconductor, „1N5222B Zener Diodes Datasheet”, Datasheet, <https://www.alldatasheet.com/html-pdf/706069/TSC/1N5222B/54/1/1N5222B.html>.
- [10] Microchip Technology Inc., „AT89S52 – 8-bit Microcontroller with 8K Bytes In-System Programmable Flash”, Datasheet, <https://ww1.microchip.com/downloads/en/DeviceDoc/doc4113.pdf>.